QUALCOMM AI RESEARCH (QARE) PREP • EYE-CARE WARM PARCHMENT EDITION

Pillar 4: NLP & Sequence Models — Intuitive Study Textbook

Tokenization (BPE/WordPiece/Unigram), Embeddings (Word2Vec/GloVe/FastText) • RNN/LSTM/GRU, CRF/CTC, Beam Search & NLP Metrics (NLP01–NLP20)

INTERACTIVE DIRECTORY • CLICK ANY TOPIC TO JUMP

Table of Contents (Mục Lục)

10 Sections Indexed

Day 9: Tokenization, Statistical NLP & Word Embeddings ..... p. 2

1.1 🌟 [Core — Bread & Butter] Subword Tokenization: BPE vs. Byte-Level BPE vs. WordPiece vs. Unigram LM ..... p. 2

1.2 🌟 [Core — Bread & Butter] N-Gram Language Models, Smoothing Hierarchy & Perplexity ..... p. 6

1.3 🌟 [Core — Bread & Butter] TF-IDF, BM25 & Levenshtein Edit Distance ..... p. 9

1.4 🌟 [Core — Bread & Butter] Static vs. Contextual Word Embeddings & Vector Space Geometry ..... p. 13

Day 10: Sequence Models (RNN/LSTM/GRU), Sequence Labeling & NLP Metrics ..... p. 17

2.1 🌟 [Core — Bread & Butter] Vanilla RNN, LSTM, GRU & BiLSTM: Equations, Parameter Counts & BPTT Gradients ..... p. 17

2.2 🌟 [Core + Deep-Dive] Sequence Labeling: BIO Tagging, Linear-Chain CRF & CTC Loss ..... p. 22

2.3 🌟 [Core — Bread & Butter] Seq2Seq, Attention Variants, Decoding & NLP Evaluation Metrics ..... p. 25

2.4 Rapid-Fire 60-Second Exam Checklist (All 20 `NLP01` – `NLP20` Targets) ..... p. 31

> ⚡ **IMPORTANT**
>
> **How to Use This Guide (Days 9–10: Nov 6 – Nov 7):** Classical and neural NLP questions on HackerRank focus heavily on **tokenization merge rules & vocabulary math**, **exact RNN/LSTM/GRU/BiLSTM parameter counts**, **Word2Vec/GloVe/FastText training objectives**, **sequence labeling (Linear-Chain CRF Viterbi & CTC collapse rules)**, **decoding algorithms (Beam Search length penalty)**, and **exact metric calculations (Perplexity, BLEU, ROUGE, TF-IDF/BM25, Levenshtein Edit Distance)**.
>
> Every section in this guide is structured in four parts:
>
> 1. 💡 **Intuition (Plain English First):** The mental model and why the method was invented, paired with an **inline visual diagram** right where the concept is introduced.
> 2. 📐 **The Math & Every Symbol Explained:** Clean display equations with every letter, subscript, and matrix dimension ($n \times m = n$ rows, m columns) defined explicitly.
> 3. 🔢 **Step-by-Step Worked Toy Example:** Concrete small-number or toy-string calculations worked out line by line.
> 4. 🎯 **Exam Takeaway & Common Traps:** The 60-second rules of thumb tested in `NLP01`–`NLP38`.

> 💡 **TIP**
>
> 🎯 **80/20 Bread & Butter Priority Map (Focus Here First!):**
>
> - 🌟 **[Core — Bread & Butter] (85% of Exam Marks — Master Cold):**
> - **Day 9:** Subword Tokenization (`BPE` raw count vs. `WordPiece` PMI merge & $|V| = |\text{Base}| + N_{\text{merges}} + |\text{Special}|$), Laplace Add-$k$ Smoothing (denom $+ k|V|$) & Perplexity ($\text{PPL} = e^{\text{NLL}_{\text{nats}}} = 2^{\text{NLL}_{\text{bits}}}$), TF-IDF / BM25 ($k_1 + 1$ asymptote, b length norm) & Levenshtein Edit Distance, Word2Vec ($2|V|d$ params, Skip-Gram vs. CBOW, SGNS $O(K)$) & Unit-Vector $L_2 = \sqrt{2(1 - \cos\theta)}$.
> - **Day 10:** RNN / GRU / LSTM / BiLSTM Gate Multipliers ($1U, 3U, 4U, 8U$ where $U = h(d + h + 1)$) & Additive Cell Highway c_t, BIO Tagging & Why CRF Beats Greedy Softmax, Seq2Seq Attention, Beam Search Length Normalization, and BLEU (Precision $\times$ BP) vs. ROUGE-L (LCS Recall).
> - 🔬 **[Advanced / Deep-Dive] (15% Bonus — Collapsed in `<details>` Blocks):**
> - Interpolated Kneser-Ney Continuation Probability, Word2Vec $3/4$ Unigram Power & Levy-Goldberg SPPMI Theorem, Formal CRF Partition Function $Z(X)$ / Viterbi DP Recurrence & CTC Alignment Collapse Rules, and ChrF++ / Self-BLEU.

Day 9: Tokenization, Statistical NLP & Word Embeddings

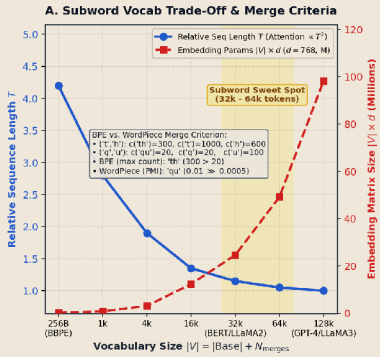
↑ Mục Lục

1.1 🌟 [Core — Bread & Butter] Subword Tokenization: BPE vs. Byte-Level BPE vs. WordPiece vs. Unigram LM

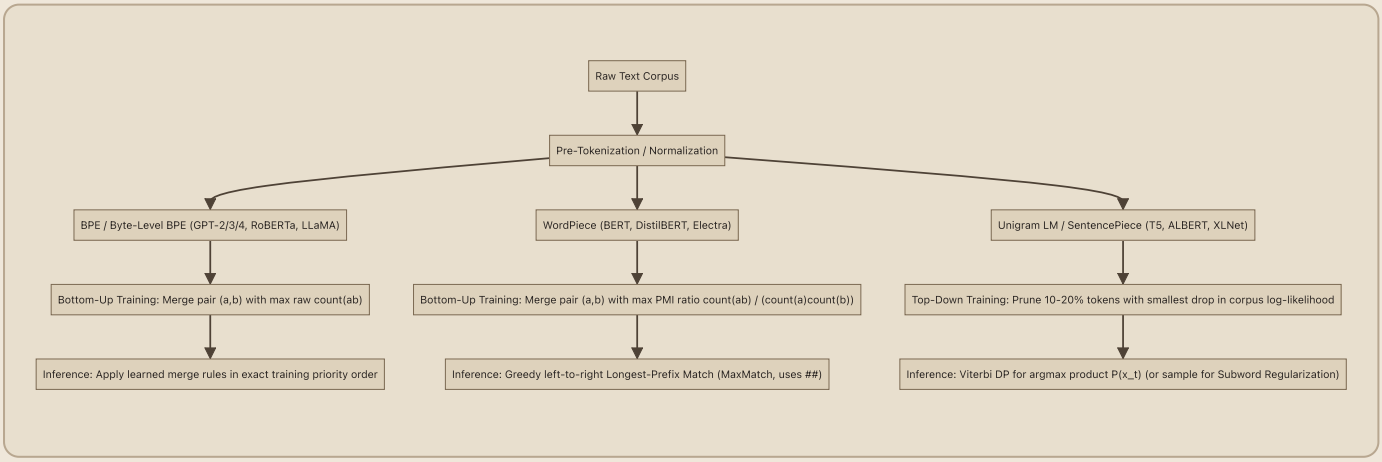
💡 Intuition (Plain English First)

Before a neural network can read text, we have to chop raw strings into discrete chunks called **tokens** and map each token to an integer ID in a vocabulary V. How big should those chunks be?

- **The Word-Level Trap:** If every distinct English word gets its own token (`"play"`, `"playing"`, `"played"`, `"unplayable"`), our vocabulary explodes past $|V| > 500,000$ words. Worse, the model has no idea that `"play"` and `"unplayable"` share a root, and any new word, typo, or rare medical term at test time becomes an unhelpful Out-Of-Vocabulary `<UNK>` token.
- **The Character-Level Trap:** What if we just tokenize into individual letters (`'p'`, `'l'`, `'a'`, `'y'`)? Now our vocabulary is tiny, and we never see `<UNK>`, *but* individual letters carry almost no meaning on their own, and our sequence length T becomes 4–6$\times$ longer! Because Transformer self-attention scales quadratically as $O(T^2)$, making a sequence $5\times$ longer makes attention $25\times$ **slower and more memory-hungry**.
- **The Subword Sweet Spot:** Subword tokenizers (**BPE**, **WordPiece**, **Unigram LM**) get the best of both worlds. Common words like `"play"` stay intact as single tokens, while rare or complex words like `"unplayable"` get split into meaningful morphemes (`"un"` + `"play"` + `"able"`).
  - **BPE (Byte-Pair Encoding)** builds its vocabulary *bottom-up* like Lego bricks: it starts with individual characters and repeatedly glues together whichever adjacent pair (a, b) appears **most frequently** in the training corpus.
  - **WordPiece** also builds *bottom-up*, but instead of picking the pair with the highest raw count, it picks the pair (a, b) that occurs together much more often than chance would predict (**Pointwise Mutual Information / PMI**). For instance, `'q'` and `'u'` may not be the most common letters overall, but whenever `'q'` shows up, `'u'` is almost guaranteed to follow!
  - **Unigram LM (SentencePiece)** works in the *exact opposite direction (top-down)*: it starts with a giant candidate list of substrings, fits a probabilistic unigram model, and iteratively **prunes (throws away)** the 10–20% of tokens whose removal hurts the overall corpus likelihood the least.



Subword Vocabulary Trade-Off & BPE vs. WordPiece Merge Criteria



📐 The Math & Every Symbol Explained

1. **Exact BPE Vocabulary Size Formula:**

$$|V|_{\text{BPE}} = |\text{Base Alphabet}| + N_{\text{merges}} + |\text{Special Tokens}|$$

- $|V|_{\text{BPE}}$: Final total number of tokens in the BPE vocabulary.
- $|\text{Base Alphabet}|$: Number of initial atomic characters (or 256 raw UTF-8 bytes in **Byte-Level BPE**).
- N_{merges}: Number of merge operations run during training. **Why?** Each merge glues an existing pair (a, b) into a brand-new token ab and adds ab to the vocabulary *without deleting* a or b (since a and b might still appear separately in other words!). Thus, 1 merge $= +1$ vocabulary token.
- $|\text{Special Tokens}|$: Control tokens added manually, such as `<pad>`, `<bos>`, `<eos>`, or `[CLS]`.

2. **BPE Training Merge Criterion (Max Raw Co-Occurrence Count):**

$$(a^*, b^*) = \arg\max_{(a,b) \in V \times V} \text{count}(ab)$$

- $\text{count}(ab)$: Total number of times token a is immediately followed by token b across all words in the training corpus.

3. **WordPiece Training Merge Criterion (Max Likelihood / PMI Ratio):**

$$(a^*, b^*) = \arg\max_{(a,b) \in V \times V} \frac{\text{count}(ab)}{\text{count}(a) \cdot \text{count}(b)}$$

- $\text{count}(a), \text{count}(b)$: Individual frequencies of token a and token b in the corpus. Dividing by $\text{count}(a) \cdot \text{count}(b)$ normalizes for how common the parts are individually, favoring strongly coupled pairs.

4. **Unigram LM (SentencePiece) Viterbi Segmentation Objective:**

$$x^* = \arg\max_{x \in S(W)} \prod_{t=1}^{|x|} P(x_t) = \arg\max_{x \in S(W)} \sum_{t=1}^{|x|} \ln P(x_t)$$

- W: The input word or string to be tokenized.
- $S(W)$: The set of all valid subword segmentations $x = (x_1, \ldots, x_{|x|})$ whose concatenation reconstructs W.
- $P(x_t)$: Learned unigram probability of subword token x_t (where $\sum_{v \in V} P(v) = 1$).

Algorithm	Used By	Training Direction	Exact Merge / Pruning Criterion	Inference Segmentation Algorithm
BPE (Byte-Pair Encoding)	GPT-2/3/4, RoBERTa, LLaMA	Bottom-up (starts with base chars/bytes)	Merges adjacent pair (a, b) with **highest raw count**: $\arg\max_{(a,b)} \text{count}(ab)$	Applies learned merge rules greedily in **exact training priority order** (NOT test-string frequency!)
WordPiece	BERT, DistilBERT, Electra	Bottom-up (uses `##` prefix for continuation subwords)	Merges pair (a, b) maximizing **PMI / Likelihood ratio**: $\arg\max_{(a,b)} \frac{\text{count}(ab)}{\text{count}(a) \cdot \text{count}(b)}$	Greedy left-to-right **Longest-Prefix Matching** (`MaxMatch`) against vocabulary V

Algorithm	Used By	Training Direction	Exact Merge / Pruning Criterion	Inference Segmentation Algorithm
Unigram LM (SentencePiece)	T5, ALBERT, XLNet, mBART	Top-down (starts with huge candidate vocab, prunes 10–20% per step)	Computes unigram probabilities via EM; **removes** tokens whose deletion causes the smallest drop in $\sum \ln P(x_i)$	**Viterbi Dynamic Programming** to find $\arg\max_{x \in S(W)} \prod_t P(x_t)$ (supports **Subword Regularization** sampling)

- **Byte-Level BPE (`BBPE` — GPT-2/4, RoBERTa, LLaMA):** Uses all 256 **raw UTF-8 bytes** as the base alphabet ($|\text{Base Alphabet}| = 256$). Because every possible Unicode string (including emojis and rare scripts) is a sequence of UTF-8 bytes, Byte-Level BPE achieves **100% open-vocabulary coverage with ZERO `<UNK>` tokens** without exploding the base alphabet to $\sim 150,000$ Unicode characters.
- **SentencePiece Lossless Space Encoding:** Treats whitespace as a normal visible meta-symbol `_` (`U+2581`, e.g., `"_Hello_world"`), bypassing language-specific whitespace pre-tokenizers (critical for languages without spaces like Chinese, Japanese, and Thai) and guaranteeing lossless detokenization `detokenize(tokenize(s)) == s`.

🔢 Step-by-Step Worked Toy Example

Part A: 2 Steps of BPE Training on `"low"`, `"lower"`, `"newest"` Suppose our toy training corpus has three words with the following frequencies (each ending with an end-of-word marker `</w>` so the model knows where words end):

- `"l o w </w>"` : count $= 5$
- `"l o w e r </w>"` : count $= 2$
- `"n e w e s t </w>"` : count $= 6$

Initial Base Alphabet: `{'l', 'o', 'w', 'e', 'r', 'n', 's', 't', '</w>'}` (9 base tokens).

- **Merge Step 1:** Count all adjacent symbol pairs across the corpus:
  - ('e', 's'): appears in `"n e w e s t </w>"` (6 times) $\implies$ count $= 6$
  - ('s', 't'): appears in `"n e w e s t </w>"` (6 times) $\implies$ count $= 6$
  - ('w', 'e'): appears in `"l o w e r </w>"` (2) + `"n e w e s t </w>"` (6) $\implies$ count $= \mathbf{8}$ *(Winner!)*
  - ('l', 'o'): appears in `"l o w </w>"` (5) + `"l o w e r </w>"` (2) $\implies$ count $= 7$
  - ('o', 'w'): appears in `"l o w </w>"` (5) + `"l o w e r </w>"` (2) $\implies$ count $= 7$
  - **Action:** Merge ('w', 'e') $\rightarrow$ 'we'. Add `'we'` to vocabulary ($|V| = 9 + 1 = 10$).
  - **Updated Corpus:** `"l o w </w>"` (5), `"l o we r </w>"` (2), `"n e we s t </w>"` (6).
- **Merge Step 2:** Recount adjacent pairs in the updated corpus:
  - ('l', 'o'): $5 + 2 = \mathbf{7}$
  - ('o', 'w'): now only in `"l o w </w>"` (5), because `"l o we r"` has `'we'` instead of `'w'`!
  - ('e', 'we'), ('we', 's'), ('s', 't'): 6 each.
  - **Action:** Merge ('l', 'o') $\rightarrow$ 'lo'. Add `'lo'` to vocabulary ($|V| = 10 + 1 = 11$).
  - **Updated Corpus:** `"lo w </w>"` (5), `"lo we r </w>"` (2), `"n e we s t </w>"` (6).

> 💡 **TIP**
>
> **Part B: Worked 30-Second BPE vs. WordPiece Comparison Trap (`NLP01`):** Suppose in a corpus: count("t") $= 1000$, count("h") $= 600$, count("th") $= 300$, whereas count("q") $= 20$, count("u") $= 100$, count("qu") $= 20$. Which pair gets merged next?
>
> - **BPE chooses `"th"`** because raw co-occurrence count count("th") $= 300 >$ count("qu") $= 20$.
> - **WordPiece chooses `"qu"`** because the PMI likelihood ratio is:
>
> $$\text{Score}_{\text{WP}}(\text{"qu"}) = \frac{\text{count("qu")}}{\text{count("q")} \cdot \text{count("u")}} = \frac{20}{20 \times 100} = 0.01 \gg \text{Score}_{\text{WP}}(\text{"th"}) = \frac{300}{1000 \times 600} = 0.0($$
>
> Why? Because `"q"` *never* appears without `"u"`, giving `"qu"` a $20\times$ higher mutual information score!

🎯 Exam Takeaway & Common Traps

- **BPE Inference Priority Trap (`NLP01`):** At test time, BPE does **not** count pair frequencies in your test sentence! It looks up its list of learned merge rules and applies them in the **exact priority order they were learned during training** (Merge Rule #1 first, then Merge Rule #2, etc.).
- **Vocabulary Math Trap (`NLP02`):** If you start with 256 bytes (Byte-Level BPE), run $30,000$ merges, and add 4 special tokens, the final vocabulary size is $256 + 30,000 + 4 = 30,260$ (never subtract old characters!).
- **WordPiece `##` Prefix:** In BERT's WordPiece, `"playing"` $\rightarrow$ `["play", "##ing"]`. The `##` means "*glue this token to the previous token without a space.*" If a test word contains even *one* unknown character that prevents `MaxMatch` from covering the full word, WordPiece replaces the **entire word** with `[UNK]`.

1.2 🌟 [Core — Bread & Butter] N-Gram Language Models, Smoothing Hierarchy & Perplexity

💡 Intuition (Plain English First)

A **Language Model (LM)** answers one question: *Given the words seen so far, what is the probability distribution of the next word?*

- **Why N-Grams Need the Markov Assumption:** Conditioning on an entire book's history $P(w_t \mid w_1, w_2, \ldots, w_{t-1})$ is impossible by counting because almost every long sentence is unique. An **N-gram model** makes a simplifying **Markov assumption**: only the last $N-1$ words matter! For a **Bigram ($N = 2$)** model, we only look at the single previous word w_{t-1} and estimate $P(w_t \mid w_{t-1})$ by simply dividing how often `"w_{t-1} w_t"` appeared by how often `"w_{t-1}"` appeared.
- **The Zero-Probability Catastrophe & Smoothing:** What if our training corpus had `"ate pizza"` and `"ate pasta"`, so count("ate") $= 100$, but at test time we see the perfectly valid phrase `"ate sushi"`, which happened to have count("ate sushi") $= 0$ in training? Raw Maximum Likelihood Estimation (MLE) assigns $P(\text{"sushi"} \mid \text{"ate"}) = \frac{0}{100} = 0$. Since sentence probability is a **product** of word probabilities, a single 0 makes the probability of the **entire test sentence equal to** 0 (and its log $P = -\infty$, so Perplexity blows up to ∞)! **Smoothing (Laplace / Add-k)** fixes this by "stealing" a tiny bit of probability mass from frequent words and giving a non-zero pseudocount k to every possible word in the vocabulary V.

- **What Perplexity (`PPL`) Really Means:** Loss functions like average Cross-Entropy (Negative Log-Likelihood, NLL) are measured in logarithmic units (*nats* or *bits*), which are hard to picture intuitively. **Perplexity** simply exponentiates the average cross-entropy loss: $\text{PPL} = e^{\text{NLL}}$. Intuitively, **Perplexity is the model's "effective branching factor"**—if a model has a Perplexity of 10, it is just as confused at each step as if it had to roll a fair 10-sided die to pick the next word! Lower Perplexity = better, more confident model.

📐 The Math & Every Symbol Explained

1. **N-Gram Chain Rule Approximation & Bigram MLE:**

$$P(w_1, w_2, \ldots, w_T) \approx \prod_{t=1}^{T} P(w_t \mid w_{t-N+1}^{t-1}), \qquad P_{\text{MLE}}(w_t \mid w_{t-1}) = \frac{\text{count}(w_{t-1}, w_t)}{\text{count}(w_{t-1})}$$

   - T: Total number of tokens in the sequence $W = (w_1, \ldots, w_T)$.
   - N: Order of the N-gram model ($N = 1$ Unigram, $N = 2$ Bigram, $N = 3$ Trigram).
   - w_{t-N+1}^{t-1}: The context window of the $N - 1$ immediately preceding tokens.
   - **Parameter Table Size:** Storing all N-gram probabilities over a vocabulary of size $|V|$ requires a conceptual table of size $|V|^N$ entries, with $|V|^{N-1}(|V| - 1)$ independent parameters (since each conditional distribution over $|V|$ next words sums to 1).

2. **Laplace (Add-1) & Add-k Smoothing Formula (`NLP03`):**

$$P_{\text{Add-}k}(w_t \mid w_{t-1}) = \frac{\text{count}(w_{t-1}, w_t) + k}{\text{count}(w_{t-1}) + k \cdot |V|}$$

   - k: Pseudocount added to every possible next word ($k = 1$ for Laplace smoothing; $0 < k < 1$ for Lidstone / Add-k smoothing).
   - $|V|$: Total number of unique token types in the vocabulary.
   - **Why is the denominator $+k|V|$ and NOT $+k$?** Because for a fixed history w_{t-1}, the conditional probabilities across all $|V|$ candidate next words $w \in V$ must sum to 1:

$$\sum_{w \in V} P_{\text{Add-}k}(w \mid w_{t-1}) = \frac{\sum_{w \in V} \text{count}(w_{t-1}, w) + \sum_{w \in V} k}{\text{count}(w_{t-1}) + k|V|} = \frac{\text{count}(w_{t-1}) + k|V|}{\text{count}(w_{t-1}) + k|V|} = 1$$

▼ 📌 **Optional Deep-Dive (Skip for Timed MCQ): Interpolated Kneser-Ney Smoothing Formula**

Why is **Interpolated Kneser-Ney** the gold standard of classical N-gram smoothing? Consider the famous "**San Francisco" Trap**: suppose the word `"Francisco"` appears 500 times in a newspaper corpus, so its raw unigram count count("Francisco") is very high. However, it *only* ever appears after the word `"San"`. If we encounter an unseen bigram after `"ate"` (like `"ate ___"`), standard backoff uses the raw unigram frequency and predicts `"ate Francisco"`!

Kneser-Ney fixes this in two ways: (1) **Absolute Discounting** subtracts a small constant $d \in (0, 1)$ (typically $d \approx 0.75$) from observed bigram counts, and (2) replaces the raw unigram backoff probability with the **Continuation Probability** $P_{\text{continuation}}(w_t)$, which measures *how many distinct preceding words w' have ever been seen before w_t*:

$$P_{\text{KN}}(w_t \mid w_{t-1}) = \frac{\max(\text{count}(w_{t-1}, w_t) - d, 0)}{\text{count}(w_{t-1})} + \lambda(w_{t-1}) P_{\text{continuation}}(w_t)$$

$$P_{\text{continuation}}(w_t) = \frac{|\{w' \in V : \text{count}(w', w_t) > 0\}|}{\sum_{w'' \in V} |\{w' \in V : \text{count}(w', w'') > 0\}|}$$

Because `"Francisco"` only ever follows 1 word (`"San"`), its continuation numerator $|\{w' : \text{count}(w', \text{"Francisco"}) > 0\}| = 1$ is tiny, preventing `"Francisco"` from polluting unseen contexts!

3. **Perplexity (`PPL`) Exact Formula (`NLP04`):**

$$\text{PPL}(W) = P(w_1, \ldots, w_T)^{-1/T} = \exp\left(-\frac{1}{T} \sum_{t=1}^{T} \ln P(w_t \mid w_{<t})\right) = \exp(\mathcal{L}_{\text{CE, nats}})$$

   - $P(w_1, \ldots, w_T)^{-1/T}$: The inverse geometric mean of the per-token probabilities assigned to the test sequence W of length T.
   - $\mathcal{L}_{\text{CE, nats}} = -\frac{1}{T} \sum_{t=1}^{T} \ln P(w_t \mid w_{<t})$: Average token-level Cross-Entropy (Negative Log-Likelihood) computed using the natural logarithm ln (base e, units = **nats**).
   - **Base-2 (Bits) Form:** If Cross-Entropy is computed using $\log_2$ (units = **bits per token**), you must exponentiate with base 2:

$$\text{PPL}(W) = 2^{\mathcal{L}_{\text{CE, bits}}}$$

   - **Uniform Baseline Identity:** A completely clueless model that guesses uniformly at random ($P(w_t \mid w_{<t}) = \frac{1}{|V|}$ for every step) achieves $\mathcal{L}_{\text{CE, nats}} = \ln |V|$ and $\text{PPL} = e^{\ln |V|} = |V|$.

🔢 Step-by-Step Worked Toy Example

Example 1: Laplace (Add-1) Smoothing Calculation (`NLP03`) Suppose our vocabulary has $|V| = 2,000$ unique words. In our training corpus, the word `"machine"` appears count("machine") = 500 times, `"machine learning"` appears 150 times, and `"machine dancing"` appears 0 times.

- **Unsmoothed MLE:**

$$P_{\text{MLE}}(\text{"learning"} \mid \text{"machine"}) = \frac{150}{500} = 0.30, \qquad P_{\text{MLE}}(\text{"dancing"} \mid \text{"machine"}) = \frac{0}{500} = 0$$

- **Laplace (Add-1, $k = 1$) Smoothing:** We add 1 to the numerator and $1 \times |V| = 2,000$ to the denominator:

$$P_{\text{Add-1}}(\text{"learning"} \mid \text{"machine"}) = \frac{150+1}{500+2000} = \frac{151}{2500} = \mathbf{0.0604}$$

$$P_{\text{Add-1}}(\text{"dancing"} \mid \text{"machine"}) = \frac{0+1}{500+2000} = \frac{1}{2500} = \mathbf{0.0004}$$

(Notice how a large vocabulary $|V| = 2,000$ pulls `"learning"` down sharply from 0.30 to 0.0604—which is why in practice people use a small $k = 0.01$ or Kneser-Ney instead of $k = 1$!)

Example 2: Perplexity from Average NLL & Token Probabilities (`NLP04`)

- **Case A (From Average NLL in Nats):** A language model evaluates a validation set and reports an average Negative Log-Likelihood loss of $\mathcal{L}_{\text{CE, nats}} = 2.0$ nats/token. What is its Perplexity?

$$\text{PPL} = e^{2.0} \approx \mathbf{7.389}$$

- **Case B (From Average NLL in Bits):** A model reports a cross-entropy of $\mathcal{L}_{\text{CE, bits}} = 3.0$ bits/token. What is its Perplexity?

$$\text{PPL} = 2^{3.0} = \mathbf{8.0}$$

- **Case C (From a 3-Token Sequence):** Suppose for a $T = 3$ token sentence, the model predicts $P(w_1) = 0.5$, $P(w_2 \mid w_1) = 0.25$, and $P(w_3 \mid w_1, w_2) = 0.125$.

$$\text{PPL} = (0.5 \times 0.25 \times 0.125)^{-1/3} = \left(\frac{1}{2} \times \frac{1}{4} \times \frac{1}{8}\right)^{-1/3} = \left(\frac{1}{64}\right)^{-1/3} = 64^{1/3} = \mathbf{4.0}$$

🎯 Exam Takeaway & Common Traps

- **Add-k Denominator Trap (`NLP03`):** Always check that the denominator is $\text{count}(w_{t-1}) + k|V|$, **never** $\text{count}(w_{t-1}) + k$ and **never** $\text{count}(w_{t-1}) + N_{\text{total tokens}}$.
- **Nats vs. Bits Trap (`NLP04`):** Check the units in the question stem! "**nats**" or "ln" $\implies e^{\mathcal{L}}$; "**bits**" or "$\log_2$" $\implies 2^{\mathcal{L}}$.
- **Cross-Tokenizer Incomparability Trap (`NLP04`):** You **cannot** compare per-token Perplexity between two models that use different tokenizers ($|V|_A = 32$k vs. $|V|_B = 128$k)! A character-level model ($|V| = 100$) will automatically have a tiny per-token PPL (~ 3), while a word-level model ($|V| = 100$k) has a much larger per-token PPL (~ 80), simply because predicting 1 letter is easier than predicting a whole word. To compare models across different tokenizers fairly, convert to **Bits-Per-Byte (BPB)**:

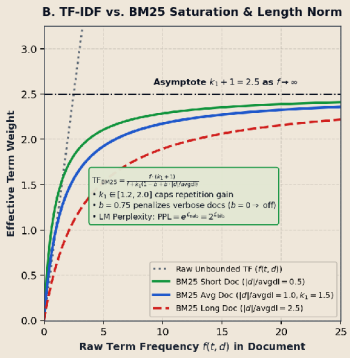
$$\text{BPB} = \frac{T_{\text{tokens}}}{T_{\text{UTF-8 bytes}}} \cdot \mathcal{L}_{\text{CE, bits}}$$

1.3 🌟 [Core — Bread & Butter] TF-IDF, BM25 & Levenshtein Edit Distance

💡 Intuition (Plain English First)

How does a search engine (or the lexical retriever inside a modern Hybrid RAG pipeline) match a user's query q against millions of documents d, and how do spell-checkers match misspelled strings?

1. **Why TF-IDF Exists:** If you search for `"the quantum compiler"`, simply counting how many query words appear in a document fails miserably: the stopword `"the"` appears 500 times in every long article, drowning out the rare, informative words `"quantum"` and `"compiler"`. **TF-IDF** multiplies two numbers for each query word t:
   - **Term Frequency (TF):** How often does word t appear in *this specific document* d? (Higher = more relevant to this document).
   - **Inverse Document Frequency (IDF):** How *rare* is word t across the entire collection of N documents? If `"the"` appears in 100% of documents, its rarity is $\log(N/N) = \log(1) = 0$, automatically wiping out stopwords!
2. **Why BM25 Beats Raw TF-IDF:** Raw TF-IDF has two big flaws:
   - *Linear TF Explosion:* If `"quantum"` appears 20 times in a document, raw TF scores it 20× higher than a document where `"quantum"` appears once—even though after seeing a word 3 or 4 times, we already know the document is about `"quantum"`! **BM25** passes term frequency through a saturating curve $\frac{f \cdot (k_1+1)}{f+k_1}$ that hits an upper ceiling of $k_1 + 1$.
   - *Long Document Bias:* A 10,000-word encyclopedic article naturally repeats words more often than a concise 200-word paragraph. **BM25** divides the term frequency by the relative document length $\frac{|d|}{\text{avgdl}}$, modulated by a hyperparameter $b \in [0, 1]$.
3. **Why Levenshtein Edit Distance Uses Dynamic Programming:** What if the user types `"INTENTION"` instead of `"EXECUTION"`? To measure how close two strings s_1 and s_2 are at the character level, **Levenshtein Edit Distance** counts the minimum number of single-character **Insertions**, **Deletions**, and **Substitutions** needed to turn s_1 into s_2. Instead of checking exponentially many alignments, the **Wagner-Fischer algorithm** builds an $(M+1) \times (N+1)$ grid where cell $D[i, j]$ solves the edit distance between the first i characters of s_1 and the first j characters of s_2 using the 3 neighboring cells above, left, and diagonal.



TF-IDF Linear Growth vs. BM25 Term Frequency Saturation & Length Normalization

📐 The Math & Every Symbol Explained

1. **TF-IDF Formula:**

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t), \qquad \text{IDF}(t) = \log\left(\frac{N}{\text{DF}(t)}\right)$$

   - $\text{TF}(t, d) = f(t, d)$: Raw count (or relative frequency) of term t in document d.
   - N: Total number of documents in the corpus.

- $\text{DF}(t)$: **Document Frequency**—the number of distinct documents in the corpus that contain term t at least once.
- *Smoothed IDF (Scikit-Learn default):* $\text{IDF}_{\text{smooth}}(t) = \ln\left(\frac{1+N}{1+\text{DF}(t)}\right) + 1$ (prevents division-by-zero for unseen words and keeps words appearing in all N documents at a baseline weight of 1).

2. **BM25 (Best Matching 25) Formula (`NLP05`):**

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}_{\text{RSJ}}(t) \cdot \frac{f(t, d) \cdot (k_1 + 1)}{f(t, d) + k_1 \cdot \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)}$$

- q: User query consisting of query terms $t \in q$.
- $f(t, d)$: Raw count of term t in document d.
- $|d|$: Length (total word count) of document d.
- avgdl: Average document length across all N documents in the corpus.
- $k_1 \in [1.2, 2.0]$: **Term Frequency Saturation** hyperparameter.
  - As $f(t, d) \to \infty$, the fraction $\frac{f(t,d)(k_1+1)}{f(t,d)+\text{const}}$ asymptotically approaches $\lim_{f \to \infty} = k_1 + 1$.
  - Setting $k_1 = 0$ turns term frequency into a binary indicator (1 if $f > 0$, else 0).
- $b \in [0, 1]$ (typically 0.75): **Document Length Normalization** hyperparameter.
  - If $|d| >$ avgdl (verbose document), the denominator grows larger, penalizing the score.
  - Setting $b = 0$ makes $1 - b + b\frac{|d|}{\text{avgdl}} = 1$, **completely disabling document length normalization**.
- $\text{IDF}_{\text{RSJ}}(t) = \ln\left(1 + \frac{N - \text{DF}(t) + 0.5}{\text{DF}(t) + 0.5}\right)$: Robertson-Spärck Jones IDF weight.

3. **Levenshtein Edit Distance Dynamic Programming Recurrence (`NLP18`):** Let string s_1 have length M and string s_2 have length N. We construct a DP matrix $D \in \mathbb{R}^{(M+1)\times(N+1)}$ ($M+1$ **rows,** $N+1$ **columns**), where row 0 and column 0 represent the empty string ϵ:
   - **Base Cases:** $D[i, 0] = i$ for $0 \le i \le M$ (i deletions to turn $s_1[1..i]$ into ϵ), and $D[0, j] = j$ for $0 \le j \le N$ (j insertions to turn ϵ into $s_2[1..j]$).
   - **Recurrence for** $1 \le i \le M$, $1 \le j \le N$**:**

$$D[i, j] = \min \begin{cases} D[i-1, j] + 1 & \text{(Deletion of } s_1[i]\text{)} \\ D[i, j-1] + 1 & \text{(Insertion of } s_2[j]\text{)} \\ D[i-1, j-1] + c_{\text{sub}}(s_1[i], s_2[j]) & \text{(Match / Substitution)} \end{cases}$$

   - $c_{\text{sub}}(s_1[i], s_2[j])$: Substitution cost, equal to 0 if $s_1[i] = s_2[j]$ (characters match!), and 1 if $s_1[i] \neq s_2[j]$ under standard Levenshtein distance (or 2 under Jurafsky & Martin's alignment convention where substitution = 1 deletion + 1 insertion).

🔢 Step-by-Step Worked Toy Example

Example 1: TF-IDF & BM25 Saturation on a 3-Document Corpus (`NLP05`) Suppose we have a corpus of $N = 3$ documents with average length avgdl $= 100$ words:

- **Doc 1 ($|d_1| = 100$):** Contains `"the"` (5 times), `"neural"` (2 times), `"compiler"` (1 time).
- **Doc 2 ($|d_2| = 100$):** Contains `"the"` (8 times), `"neural"` (1 time), `"compiler"` (0 times).
- **Doc 3 ($|d_3| = 100$):** Contains `"the"` (4 times), `"neural"` (0 times), `"compiler"` (0 times).

1. **Compute Unsmoothed Base-10 IDF(t) $= \log_{10}(N/\text{DF}(t))$:**
   - `"the"` appears in all 3 docs (DF $= 3$): IDF("the") $= \log_{10}(3/3) = \log_{10}(1) = \mathbf{0}$. Even though `"the"` appears 8 times in Doc 2, its TF-IDF $= 8 \times 0 = \mathbf{0}$!
   - `"neural"` appears in 2 docs (DF $= 2$): IDF("neural") $= \log_{10}(3/2) \approx \mathbf{0.176}$.
   - `"compiler"` appears in only 1 doc (DF $= 1$): IDF("compiler") $= \log_{10}(3/1) \approx \mathbf{0.477}$.
2. **Watch BM25 Saturation in Action ($k_1 = 1.5$, $|d| = \text{avgdl}$):** Since $|d| = \text{avgdl} = 100$, the length factor is $1 - b + b(1) = 1$, so the BM25 TF weight is $\frac{f \cdot (1.5+1)}{f+1.5} = \frac{2.5f}{f+1.5}$:
   - If $f = 1$: BM25 TF weight $= \frac{2.5(1)}{1+1.5} = \mathbf{1.00}$.
   - If $f = 6$: BM25 TF weight $= \frac{2.5(6)}{6+1.5} = \frac{15}{7.5} = \mathbf{2.00}$ (a $6\times$ increase in word count only doubles the score!).
   - If $f = 1000$: BM25 TF weight $= \frac{2.5(1000)}{1001.5} \approx \mathbf{2.496} \leq k_1 + 1 = \mathbf{2.50}$ (hard ceiling at 2.50!).

Example 2: Levenshtein Edit Distance DP Table (`NLP18`) Let's transform $s_1 =$ "CAT" ($M = 3$) into $s_2 =$ "CUT" ($N = 3$) and also check the classic exam pair `"INTENTION"` $\rightarrow$ `"EXECUTION"`:

- For `"CAT"` $\rightarrow$ `"CUT"`, we build a $(3+1) \times (3+1) = 4 \times 4$ matrix (4 rows, 4 columns):
  - Row 1 (`'C'` vs `'C','U','T'`): `'C'` matches `'C'` ($D[1,1] = D[0,0] + 0 = 0$), then $D[1,2] = 1, D[1,3] = 2$.
  - Row 2 (`'A'` vs `'C','U','T'`): `'A'` $\neq$ `'U'`, so $D[2,2] = \min(D[1,2]+1, D[2,1]+1, D[1,1]+1) = \min(2, 2, 0+1) = \mathbf{1}$ (1 substitution `'A'` $\rightarrow$ `'U'`).
  - Row 3 (`'T'` vs `'C','U','T'`): `'T'` matches `'T'`, so $D[3,3] = D[2,2] + 0 = \mathbf{1}$.
- **Canonical Exam Pair (`'INTENTION'` $\rightarrow$ `'EXECUTION'`, `NLP18`):** Both strings have length 9 and share the identical 4-character suffix `'TION'` (0 edits needed for the last 4 characters!). We only need to transform the 5-character prefix `'INTEN'` into `'EXECU'`:
  - Under **standard Levenshtein distance (substitution cost $= 1$)**: Replacing all 5 prefix positions takes at most 5 **edits** (specifically: delete `'I'`, `'N'`$\rightarrow$`'E'`, `'T'`$\rightarrow$`'X'`, insert `'E'`, `'N'`$\rightarrow$`'C'`, `'U'`? Even simpler: 5 direct position operations $= 5$ **edits**).
  - Under **Jurafsky & Martin's alternate alignment cost (substitution cost $= 2$):** Because `'INTEN'` and `'EXECU'` share one `'E'` that aligns, 1 deletion (`'I'`) + 1 insertion (`'C'`) + 3 substitutions ($3 \times 2 = 6$) $= 8$ **edits**.

🎯 Exam Takeaway & Common Traps

- **Zero-IDF Stopword Rule:** In unsmoothed TF-IDF, any word appearing in **all N documents** has DF(t) $= N \implies$ IDF(t) $= \log(N/N) = \mathbf{0}$.
- **BM25 Asymptote Trap (`NLP05`):** If an MCQ asks for the maximum possible term-frequency multiplier in BM25 as term count $f(t,d) \rightarrow \infty$ (for an average-length document), the answer is $k_1 + 1$ (NOT ∞ and NOT k_1).
- **Edit Distance Cost Convention (`NLP18`):** Always read whether the problem specifies **unit substitution cost (1)** (Edit("INTENTION","EXECUTION") $= 5$) or **substitution cost $= 2$** ($= 8$). Time complexity is always $O(M \cdot N)$, and space complexity can be optimized from $O(M \cdot N)$ to $O(\min(M, N))$ if you only need the final distance value (since row i only depends on row $i-1$).

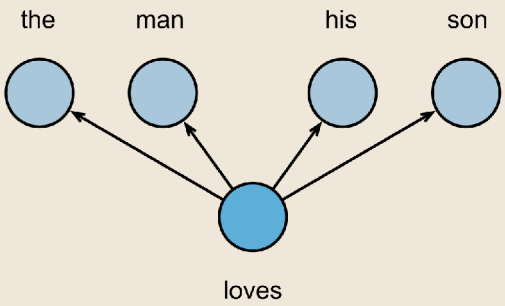
1.4 🌟 [Core — Bread & Butter] Static vs. Contextual Word Embeddings & Vector Space Geometry

💡 Intuition (Plain English First)

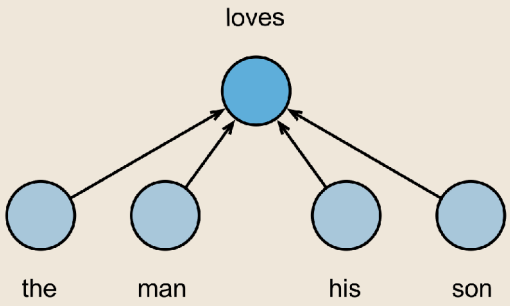
One-hot vectors treat every word as completely orthogonal: in a one-hot vocabulary of size $|V| = 50,000$, the dot product between `"cat"` and `"kitten"` is 0—just as unrelated as `"cat"` and `"refrigerator"`. How do we learn dense vectors in $\mathbb{R}^d$ (say $d = 300$) where semantically similar words point in similar directions?

1. **The Distributional Hypothesis & Word2Vec:** Linguist J.R. Firth famously said, "*You shall know a word by the company it keeps.*" Words like `"cat"` and `"kitten"` appear surrounded by the same neighbor words (`"pet"`, `"purr"`, `"milk"`, `"vet"`). **Word2Vec (Mikolov et al., 2013)** turns unsupervised text into a self-supervised prediction game over a sliding window of size m:
   - **CBOW (Continuous Bag-of-Words):** Takes the surrounding $2m$ context words, **averages** their vectors together, and tries to predict the missing **center word** in the middle (like a fill-in-the-blank quiz). Because it averages the context into 1 vector, CBOW runs **1 update per window** (fast!) and smooths over noise, making it great for **frequent words**.
   - **Skip-Gram:** Does the exact reverse—takes the **single center word** w_c and tries to independently predict each of the $2m$ **surrounding context words** one by one ($2m$ updates per window). Because a rare word doesn't get averaged away with its neighbors and gets $2m$ direct gradient updates every time it appears, Skip-Gram is much better for **rare words**.
2. **Why Word2Vec Needs TWO Matrices ($2|V|d$ Parameters):** Why doesn't Word2Vec just use a single $|V| \times d$ matrix? If a word w used the exact same vector v_w when acting as a center word and when acting as a context word, the model would need $v_w^T v_w = \|v_w\|_2^2$ (which is huge!) in the softmax denominator, representing the absurd event of a word appearing next to *itself*! To keep the math clean and stable, Word2Vec maintains **two separate lookup tables**: a **Center (Input) matrix** W_{in} and a **Context (Output) matrix** W_{out}.
3. **Why Full Softmax Is Too Slow & How SGNS Fixes It:** Computing a standard Softmax over $|V| = 100,000$ words requires computing $100,000$ dot products in the denominator for *every single training step*!
   - **Hierarchical Softmax** arranges all $|V|$ words as leaves of a binary Huffman tree, turning a $100,000$-way choice into $\approx \log_2(100,000) \approx 17$ binary left/right coin flips along a tree path.
   - **Skip-Gram with Negative Sampling (SGNS)** is even simpler: instead of asking "*Which of the 100,000 words is the true neighbor?*", it asks a binary yes/no question: "*Is* `(cat, purr)` *a real pair from the text (label 1), and are these* $K = 5$ *randomly drawn fake pairs* `(cat, carburetor)`, `(cat, democracy)`, ... *noise (label 0)?*" That reduces the cost from $O(|V|)$ to just $O(K)$ (6 dot products instead of $100,000$)!
4. **From Static Word2Vec to FastText & Contextual BERT:** Word2Vec and GloVe assign **1 fixed vector per word type** in a static lookup table. That creates two huge problems: (1) any unseen typo or new word at test time fails (`<UNK>`), and (2) **polysemy**—the word `"bank"` gets the exact same vector in `"river bank"` and `"investment bank"`!
   - **FastText** fixes the `<UNK>` / morphology problem by breaking every word into **character** n**-grams** (`"<wh"`, `"whe"`, `"her"`, `"ere"`, `"re>"`) and summing their vectors—so even a misspelled or unseen word gets a vector!
   - **ELMo & BERT** fix the **polysemy** problem by running the full sentence through a deep bidirectional encoder (BiLSTM in ELMo, Transformer in BERT) so the representation h_t of `"bank"` dynamically

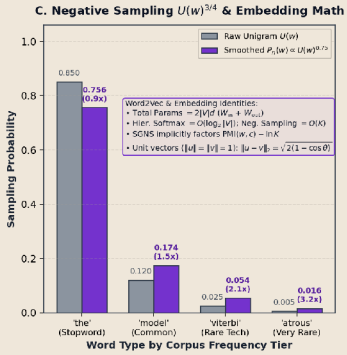
changes depending on the rest of the sentence.



Word2Vec Skip-Gram: Center Word Predicts Surrounding Context Words



Word2Vec CBOW: Context Words Predict the Center Word



Word2Vec 2|V|d Architecture & Skip-Gram with Negative Sampling (SGNS)

📐 The Math & Every Symbol Explained

1. **Word2Vec Parameter Count & Full Softmax Objective (`NLP06`):**

$$\text{Total Learnable Parameters} = |W_{\text{in}}| + |W_{\text{out}}| = |V| \cdot d + |V| \cdot d = \mathbf{2|V|d}$$

$$P(w_o \mid w_c) = \frac{\exp(u_o^T v_c)}{\sum_{w=1}^{|V|} \exp(u_w^T v_c)}$$

- $|V|$: Vocabulary size (number of unique words).
- d: Embedding dimension (e.g., $d = 300$).
- $W_{\text{in}} \in \mathbb{R}^{|V| \times d}$ ($|V|$ **rows,** d **columns**): Center/Input embedding matrix; row c is the center word vector $v_c \in \mathbb{R}^d$.

- $W_{\text{out}} \in \mathbb{R}^{|V| \times d}$ ($|V|$ **rows,** d **columns**): Context/Output embedding matrix; row o is the context word vector $u_o \in \mathbb{R}^d$.
- *(Note: Word2Vec linear projections have* ***no bias terms****! After training,* W_{in} *is either used alone or averaged/concatenated with* W_{out}*).*

2. **Skip-Gram with Negative Sampling (SGNS) Objective (`NLP07`):**

$$\mathcal{J}_{\text{SGNS}}(w_c, w_o) = \ln \sigma(u_o^T v_c) + \sum_{k=1}^{K} \mathbb{E}_{w_k \sim P_n(w)} \left[\ln \sigma(-u_k^T v_c) \right]$$

- $\sigma(z) = \frac{1}{1+e^{-z}}$: Standard logistic sigmoid function (note that $\sigma(-z) = 1 - \sigma(z)$).
- $u_o^T v_c$: Dot product between true context vector u_o and center vector v_c (we want this to be large and positive so $\sigma(u_o^T v_c) \to 1$).
- K: Number of negative (noise) samples drawn per positive pair (typically $K \in [5, 20]$ for small datasets, $K \in [2, 5]$ for massive datasets).
- u_k: Context vector of the k-th negative sample w_k drawn from noise distribution $P_n(w)$ (we want $u_k^T v_c$ to be negative so $\sigma(-u_k^T v_c) \to 1$).

▼ 📌 **Optional Deep-Dive (Skip for Timed MCQ): 3/4 Unigram Noise Power & Levy-Goldberg SPPMI Theorem**

- **Why does Word2Vec sample negative words from** $P_n(w) \propto U(w)^{3/4}$ **instead of raw unigram frequency** $U(w)$**?** If we sampled noise words proportionally to raw frequency $U(w)$, ultra-frequent stopwords like `"the"` would dominate the negative samples, while rare words would almost never be trained as negative examples. Raising every probability to the 0.75 power dampens big numbers and gives rare words a huge relative boost! For example, compare a common word ($U(\text{"is"}) = 0.05$) with a rare word ($U(\text{"aardvark"}) = 0.0001$):
  - Raw ratio: $0.05 / 0.0001 = 500\times$ more likely.
  - Smoothed 3/4-power ratio: $0.05^{0.75} / 0.0001^{0.75} = 0.1057 / 0.001 = 105.7\times$—giving `"aardvark"` nearly a $5\times$ **boost** in sampling probability!
- **Levy & Goldberg (2014) Theorem:** Optimizing Skip-Gram with K Negative Samples is mathematically equivalent to implicitly factorizing the **Shifted Pointwise Mutual Information (SPPMI)** matrix:

$$v_c^T u_o = \text{PMI}(w_c, w_o) - \ln K = \ln \frac{P(w_c, w_o)}{P(w_c)P(w_o)} - \ln K$$

3. **Unit-Normalized Cosine Similarity vs. Euclidean (L_2) Distance Identity (`NLP08`):** For any two vectors $u, v \in \mathbb{R}^d$ that have been L_2-normalized to unit length ($\|u\|_2 = \|v\|_2 = 1$):

$$\cos\theta = \frac{u^T v}{\|u\|_2 \|v\|_2} = u^T v$$

$$\|u - v\|_2^2 = \|u\|_2^2 + \|v\|_2^2 - 2u^T v = 1 + 1 - 2\cos\theta = 2(1 - \cos\theta) \implies \|u - v\|_2 = \sqrt{2(1 - \cos\theta)}$$

- Because $\sqrt{2(1-\cos\theta)}$ is a strictly decreasing function of $\cos\theta$ on $[-1, 1]$, **ranking unit-normalized embeddings by smallest Euclidean distance** $\|u - v\|_2$ **produces the exact same nearest-neighbor ranking as sorting by largest Cosine Similarity** $\cos\theta$**!**

Model	Granularity	Handles Unseen OOV Words?	Polysemy (`"river bank"` vs. `"investment bank"`)?	Core Mechanism / Objective
Word2Vec	Word type	No (`<UNK>`)	No (1 static vector per word)	Local context window prediction (SGNS / CBOW)
GloVe	Word type	No (`<UNK>`)	No (1 static vector per word)	Global log-bilinear regression on co-occurrence counts $\ln X_{ij}$
FastText	Character n-grams ($n \in [3, 6]$)	**Yes!** (sums subword n-grams)	No (static sum of char n-grams)	Represents word as $v_w + \sum_{g \in \mathcal{G}(w)} z_g$ (ideal for typos & rich morphology)
ELMo / BERT	Subword + Full Sequence	Yes (via subwords)	**Yes!** (contextual hidden state h_t)	Deep bidirectional sequence encoding (BiLSTM LM in ELMo; Masked LM in BERT)

> 📌 **NOTE**
>
> **BERT Masked Language Modeling (MLM) 80-10-10 Rule (`NLP19`):** During BERT pretraining, 15% of tokens in each sequence are selected for MLM loss. Of those selected tokens: 80% **are replaced with** `[MASK]`, 10% **are replaced with a random vocabulary token**, and 10% **are kept unchanged**. Why not 100% `[MASK]`? Because the `[MASK]` token never appears during downstream fine-tuning (creating a **pretrain-finetune discrepancy**)! If BERT only ever had to predict `[MASK]` tokens, it could lazily ignore non-masked tokens. Including 10% random and 10% unchanged tokens forces the Transformer encoder to build rich, denoised contextual representations for *every* input token, because any token might be evaluated in the loss!

🔢 Step-by-Step Worked Toy Example

Example 1: Word2Vec Parameter Count & Per-Step Softmax Speedup (`NLP06` , `NLP07`) Suppose we train a Skip-Gram model on a vocabulary of size $|V| = 65,536 = 2^{16}$ with embedding dimension $d = 300$ and $K = 5$ negative samples:

1. **Total Learnable Parameters:**

$$\text{Params} = 2 \times |V| \times d = 2 \times 65,536 \times 300 = \mathbf{39,321,600} \quad (\text{NOT } 65,536 \times 300 = 19.66\text{M!})$$

2. **Dot Products per Context Prediction Step:**
   - **Full Softmax:** $O(|V|) = \mathbf{65,536}$ dot products.
   - **Hierarchical Softmax:** $O(\log_2 |V|) = \log_2(2^{16}) = \mathbf{16}$ dot products ($4,096\times$ speedup!).
   - **Negative Sampling ($K = 5$):** $O(K + 1) = 5 + 1 = \mathbf{6}$ dot products ($10,922\times$ speedup!).

Example 2: Unit-Vector Euclidean Distance from Cosine Similarity (`NLP08`) Two unit-normalized word embeddings $u, v \in \mathbb{R}^{768}$ ($\|u\|_2 = \|v\|_2 = 1$) have a cosine similarity of $\cos\theta = 0.68$. What is their exact Euclidean distance $\|u - v\|_2$?

- Step 1: Plug into the squared L_2 identity:

$$\|u - v\|_2^2 = 2(1 - \cos\theta) = 2(1 - 0.68) = 2(0.32) = 0.64$$

- Step 2: Take the square root:

  $$\|u - v\|_2 = \sqrt{0.64} = \mathbf{0.80}$$

- *(Bonus check: What if u and v are orthogonal, $\cos\theta = 0$? Then $\|u - v\|_2 = \sqrt{2(1-0)} = \sqrt{2} \approx 1.414$. What if they point in opposite directions, $\cos\theta = -1$? Then $\|u - v\|_2 = \sqrt{2(1-(-1))} = \sqrt{4} = 2.0$.)*

🎯 Exam Takeaway & Common Traps

- **The $2|V|d$ Parameter Trap (`NLP06`):** Whenever an exam asks for the number of trainable parameters in Word2Vec (Skip-Gram or CBOW), remember to multiply $|V| \times d$ by 2 because of the separate input (W_{in}) and output (W_{out}) embedding matrices!
- **CBOW vs. Skip-Gram Rare Word Trap: Skip-Gram** is superior for **rare words** (no context averaging; $2m$ independent updates per window), whereas **CBOW** trains faster and is slightly better for **frequent words**.
- **FastText vs. BERT Polysemy Trap:** FastText solves **OOV / typos** via character n-grams, but it is still a **static** embedding model—it assigns the exact same vector to `"bank"` regardless of context! Only **contextual encoders (ELMo, BERT, GPT)** resolve **polysemy**.

Day 10: Sequence Models (RNN/LSTM/GRU), Sequence Labeling & NLP Metrics

↑ Mục Lục

2.1 🌟 [Core — Bread & Butter] Vanilla RNN, LSTM, GRU & BiLSTM: Equations, Parameter Counts & BPTT Gradients

💡 Intuition (Plain English First)

How do we process a variable-length sentence $x_1, x_2, \ldots, x_T$ while keeping a memory of what came earlier?

1. **Why Vanilla RNN Gradients Vanish or Explode:** A **Vanilla RNN** maintains a hidden state vector h_t and updates it at every step by multiplying the previous state h_{t-1} by a weight matrix W_{hh}, adding the new input $W_{xh}x_t$, and squashing the result through tanh:

   $$h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$$

   Notice what happens during **Backpropagation Through Time (BPTT)**: to send an error signal from the end of a 50-word sentence ($t = 50$) back to the first word ($t = 1$), the chain rule multiplies by the Jacobian $\frac{\partial h_k}{\partial h_{k-1}} = \text{diag}(1 - h_k^2)W_{hh}^T$ **50 times in a row**!

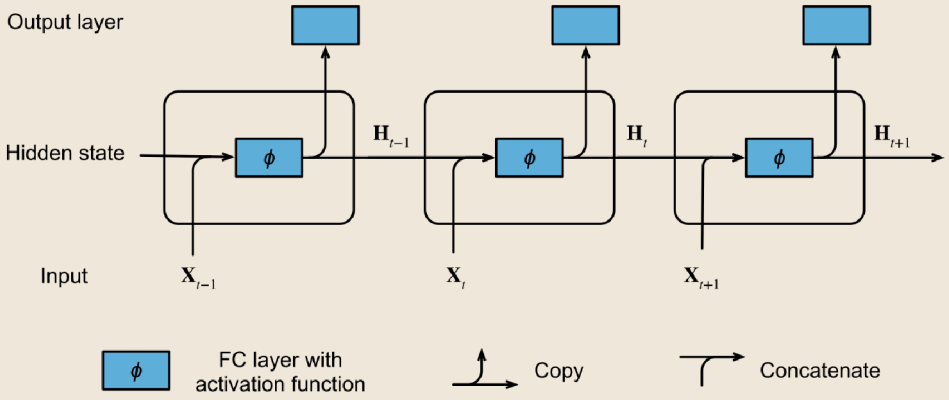
   - Just like multiplying $0.8^{50} = 0.000014$, if the largest singular value $\sigma_{\max}(W_{hh}) < 1$ (or if tanh saturates so $1 - h_k^2 \to 0$), the gradient **vanishes exponentially** to zero—the network completely forgets the subject at the start of the sentence!
   - Conversely, like multiplying $1.2^{50} = 9,100$, if $\sigma_{\max}(W_{hh}) > 1$, the gradient **explodes exponentially** to `NaN`!

2. **How LSTM's Additive Cell State Creates a "Gradient Highway":** Why did **Long Short-Term Memory (LSTM, Hochreiter & Schmidhuber 1997)** solve the vanishing gradient problem? Instead of forcing the memory through a matrix multiplication and a squashing tanh at every single step, LSTM introduces a separate **Cell State conveyor belt** c_t whose update is **purely element-wise addition**:

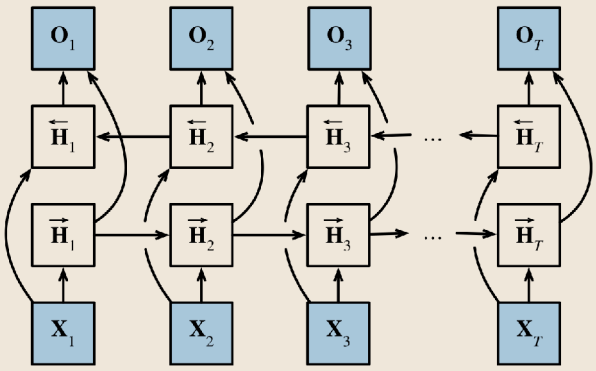
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

Look at the derivative along this highway: $\frac{\partial c_t}{\partial c_{t-1}} = f_t$ (plus indirect terms through the gates). There is **no matrix multiplication** W_{hh} and **no** $\tanh'$ **squashing factor** blocking the main path! Whenever the **forget gate** $f_t \approx 1$, the error gradient flows straight back across 100+ time steps completely undiminished (just like residual connections $y = x + F(x)$ in ResNets!).

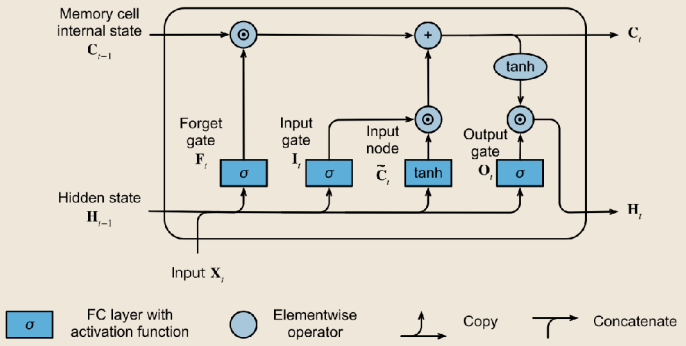
3. **How Each LSTM Gate Works (The Mental Model):** Every gate is a vector of numbers between 0 and 1 (produced by σ) that acts like a dimmer switch via element-wise multiplication $\odot$:
   - **Forget Gate ($f_t \in (0,1)^h$):** "*What fraction of my old long-term memory c_{t-1} should I keep vs. erase?*" (e.g., when a sentence switches to a new subject, $f_t \to 0$ to wipe out the old subject's gender/number).
   - **Input Gate ($i_t \in (0,1)^h$) & Candidate Memory ($\tilde{c}_t \in (-1,1)^h$):** "*What new information $\tilde{c}_t$ is arriving right now, and how much of it (i_t) should I write onto the conveyor belt?*"
   - **Output Gate ($o_t \in (0,1)^h$):** "*Out of everything stored in my long-term cell state c_t, what part is relevant to expose right now as my short-term working state $h_t = o_t \odot \tanh(c_t)$?*"
4. **How GRU Simplifies LSTM: Gated Recurrent Unit (Cho et al., 2014)** asks: *Do we really need two separate state vectors (c_t and h_t) and four gates?* GRU merges c_t and h_t into a **single state** h_t and uses only **3 gate blocks** (**Reset gate** r_t, **Update gate** z_t, and **Candidate** $\tilde{h}_t$), coupling the forget and input decisions together via a convex combination $h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$. This cuts parameter count and memory by 25% compared to LSTM while keeping the additive gradient highway!



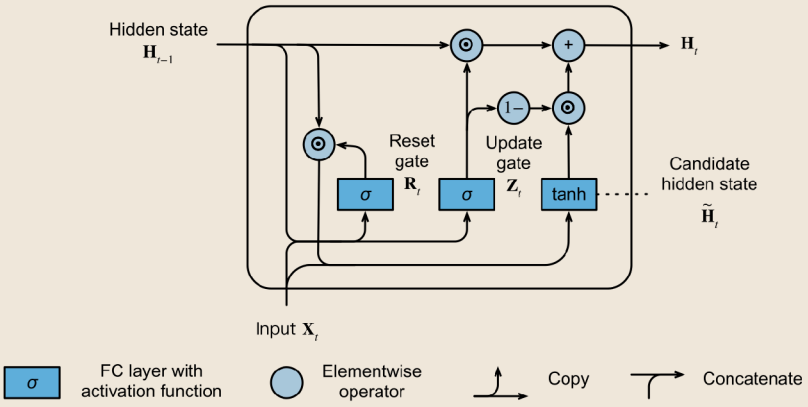
Vanilla RNN Hidden State Recurrence



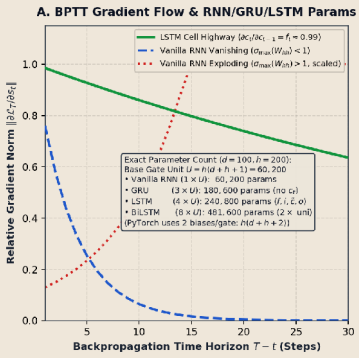
Bidirectional RNN Forward & Backward State Concatenation



LSTM Cell State Highway & 4 Gating Units (Forget, Input, Candidate, Output)



GRU Reset & Update Gating Cell



BPTT Gradient Flow & RNN / GRU / LSTM / BiLSTM Gate Parameter Multipliers

📐 The Math & Every Symbol Explained

Let input at step t be $x_t \in \mathbb{R}^d$ ($d \times 1$ column vector) and hidden state be $h_t \in \mathbb{R}^h$ ($h \times 1$ column vector). When we concatenate $[h_{t-1}; x_t]$, we get a vector of length $h + d$.

1. **Single-Gate Parameter Block Size (U):** Every gate takes $x_t \in \mathbb{R}^d$ and $h_{t-1} \in \mathbb{R}^h$ and outputs a vector in $\mathbb{R}^h$. This requires:
   - Input-to-hidden weight matrix $W_{xh} \in \mathbb{R}^{h \times d}$ (h **rows,** d **columns** $\implies h \cdot d$ parameters).
   - Hidden-to-hidden weight matrix $W_{hh} \in \mathbb{R}^{h \times h}$ (h **rows,** h **columns** $\implies h \cdot h = h^2$ parameters).
   - Bias vector $b \in \mathbb{R}^h$ (h **rows,** 1 **column** $\implies h$ parameters in standard textbook formulation; or **two** bias vectors $b_{ih}, b_{hh} \in \mathbb{R}^h \implies 2h$ parameters in PyTorch's `nn.RNN` / `nn.GRU` / `nn.LSTM` implementation):

$$U_{\text{1-bias}} = h(d+h+1), \qquad U_{\text{PyTorch (2-bias)}} = h(d+h+2)$$

2. **LSTM Exact Gate Equations (4 Gate Blocks $\implies 4 \times U$ Parameters, `NLP10`):**

$$f_t = \sigma\left(W_f[h_{t-1}; x_t] + b_f\right) \qquad (\text{Forget Gate, } f_t \in \mathbb{R}^h)$$

$$i_t = \sigma\left(W_i[h_{t-1}; x_t] + b_i\right) \qquad (\text{Input Gate, } i_t \in \mathbb{R}^h)$$

$$\tilde{c}_t = \tanh\left(W_c[h_{t-1}; x_t] + b_c\right) \qquad (\text{Candidate Cell State, } \tilde{c}_t \in \mathbb{R}^h)$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \qquad (\text{Additive Cell State Highway, } c_t \in \mathbb{R}^h)$$

$$o_t = \sigma\left(W_o[h_{t-1}; x_t] + b_o\right) \qquad (\text{Output Gate, } o_t \in \mathbb{R}^h)$$

$$h_t = o_t \odot \tanh(c_t) \qquad (\text{Exposed Hidden State, } h_t \in \mathbb{R}^h)$$

- $W_f, W_i, W_c, W_o \in \mathbb{R}^{h \times (h+d)}$ (h **rows,** $h+d$ **columns**): Combined weight matrices for the 4 gates.
- $\odot$: Hadamard (element-wise) vector multiplication.
- **Positive Forget-Gate Bias Trick:** Initializing $b_f = +1.0$ (Jozefowicz et al., 2015) makes $f_t \approx \sigma(1.0) \approx 0.73$ at the start of training, ensuring the cell state remembers long-range dependencies by default instead of immediately zeroing them out.

3. **GRU Exact Gate Equations (3 Gate Blocks $\implies 3 \times U$ Parameters):**

$$r_t = \sigma\left(W_r[h_{t-1}; x_t] + b_r\right) \qquad (\text{Reset Gate, } r_t \in \mathbb{R}^h)$$

$$z_t = \sigma\left(W_z[h_{t-1}; x_t] + b_z\right) \qquad (\text{Update Gate, } z_t \in \mathbb{R}^h)$$

$$\tilde{h}_t = \tanh\left(W_h[r_t \odot h_{t-1}; x_t] + b_h\right) \qquad (\text{Candidate Hidden State, } \tilde{h}_t \in \mathbb{R}^h)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t \qquad (\text{Interpolated Hidden State, } h_t \in \mathbb{R}^h)$$

4. **BPTT Gradient Flow Equation (`NLP11`):**

$$\frac{\partial \mathcal{L}_T}{\partial h_1} = \frac{\partial \mathcal{L}_T}{\partial h_T} \prod_{k=2}^{T} \frac{\partial h_k}{\partial h_{k-1}} = \frac{\partial \mathcal{L}_T}{\partial h_T} \prod_{k=2}^{T} \text{diag}(1 - h_k^2) W_{hh}^T$$

- **Global Gradient Norm Clipping (Fix for Exploding Gradients):** If $\|g\|_2 > \tau$, rescale the entire gradient vector $g \leftarrow \frac{\tau}{\|g\|_2} g$ so its direction is preserved while its norm is capped at threshold τ.

Architecture	Gates / States	Theoretical Params (1 bias/gate)	PyTorch Params (2 biases/gate)	Worked Example ($d=100, h=200$, 1 bias, `NLP09`)
Vanilla RNN	1 (tanh state h_t)	$1 \times h(d+h+1)$	$1 \times h(d+h+2)$	$200(100+200+1) = \mathbf{60,200}$

Architecture	Gates / States	Theoretical Params (1 bias/gate)	PyTorch Params (2 biases/gate)	Worked Example ($d = 100, h = 200$, 1 bias, `NLP09`)
GRU	3 ($r_t, z_t, \tilde{h}_t$; no c_t)	$3 \times h(d+h+1)$	$3 \times h(d+h+2)$	$3 \times 60,200 = \mathbf{180,600}$
LSTM	4 ($f_t, i_t, \tilde{c}_t, o_t$; has c_t, h_t)	$4 \times h(d+h+1)$	$4 \times h(d+h+2)$	$4 \times 60,200 = \mathbf{240,800}$ ($241,600$ in PyTorch)
BiLSTM (1 Layer)	$2 \times 4 = 8$ ($\overrightarrow{\text{LSTM}} + \overleftarrow{\text{LSTM}}$)	$8 \times h(d+h+1)$	$8 \times h(d+h+2)$	$2 \times 240,800 = \mathbf{481,600}$ (output dim $= 2h = 400$)

🔢 Step-by-Step Worked Toy Example

Let's walk through a tiny toy network with input dimension $d = 4$ and hidden state dimension $h = 8$ (using standard textbook 1 bias per gate) to see how every single parameter is counted:

1. **Step 1: Compute 1 Gate Block U:**
   - Input-to-hidden matrix W_{xh} has shape 8×4 (8 rows, 4 columns) $= 32$ weights.
   - Hidden-to-hidden matrix W_{hh} has shape 8×8 (8 rows, 8 columns) $= 64$ weights.
   - Bias vector b has shape $8 \times 1 = 8$ biases.
   - **Single-Gate Block Total:** $U = 32 + 64 + 8 = 8(4+8+1) = 8 \times 13 = \mathbf{104}$ parameters.
2. **Step 2: Multiply by Number of Gates:**
   - **Vanilla RNN (1 gate):** $1 \times 104 = \mathbf{104}$ parameters.
   - **GRU (3 gates: $r_t, z_t, \tilde{h}_t$):** $3 \times 104 = \mathbf{312}$ parameters.
   - **LSTM (4 gates: $f_t, i_t, \tilde{c}_t, o_t$):** $4 \times 104 = \mathbf{416}$ parameters *(in PyTorch with 2 biases/gate:* $4 \times 8(4+8+2) = \mathbf{448}$ *parameters)*.
   - **1-Layer Bidirectional LSTM (Forward LSTM + Backward LSTM):** $2 \times 416 = \mathbf{832}$ parameters. At each time step t, the forward LSTM outputs $\overrightarrow{h}_t \in \mathbb{R}^8$ and the backward LSTM outputs $\overleftarrow{h}_t \in \mathbb{R}^8$, which are concatenated into $h_t^{(1)} = [\overrightarrow{h}_t; \overleftarrow{h}_t] \in \mathbb{R}^{\mathbf{16}}$.
3. **Step 3: What if we stack a 2nd BiLSTM Layer on top? (The Stacked BiLSTM Trap!)** Because Layer 1 outputs a concatenated vector of size $2h = 16$, **Layer 2's input dimension is $d_2 = 2h = 16$ (NOT 4 and NOT 8!)**:
   - Layer 2 single-gate block: $U_2 = h(2h+h+1) = 8(16+8+1) = 8 \times 25 = 200$.
   - Layer 2 BiLSTM (8 gates total): $8 \times 200 = \mathbf{1,600}$ parameters!
   - Total 2-Layer BiLSTM parameters $= 832$ (Layer 1) $+ 1,600$ (Layer 2) $= \mathbf{2,432}$ parameters.

🎯 Exam Takeaway & Common Traps

- **Memorize the Multiplier Rule (`NLP09`):** Once you compute $U = h(d+h+1)$, **RNN** $= 1U$, **GRU** $= 3U$, **LSTM** $= 4U$, **1-Layer BiLSTM** $= 8U$. Always check if a PyTorch-specific question gives an option that is $+4h$ larger for LSTM (due to PyTorch's redundant second bias b_{hh}).
- **Stacked BiLSTM Layer-2 Input Trap:** In a multi-layer BiLSTM with hidden size h per direction, Layer 1 outputs dimension $2h$, so Layer 2 has input size $d_2 = 2h$!
- **Vanishing vs. Exploding Fix Trap (`NLP11`): Gradient Norm Clipping fixes exploding gradients ONLY**—it does *nothing* to help vanishing gradients! Vanishing gradients require architectural changes (**LSTM/GRU**

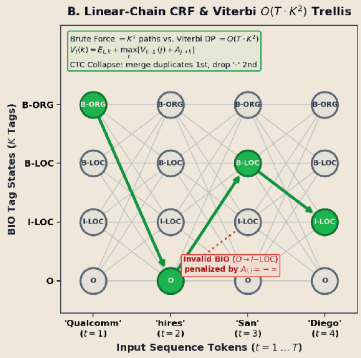
additive cell state updates, **residual connections**, or **Transformer direct attention paths**).

2.2 🌟 [Core + Deep-Dive] Sequence Labeling: BIO Tagging, Linear-Chain CRF & CTC Loss

💡 Intuition (Plain English First)

In sequence labeling tasks like Named Entity Recognition (NER), Part-of-Speech (POS) tagging, or Automatic Speech Recognition (ASR), we want to assign a sequence of labels to a sequence of inputs.

1. **Why We Need BIO Tagging & Span-Level F1:** Suppose a sentence says `"Per-Olof flew from New York Los Angeles yesterday"` (with a missing `"to"`). If we only labeled words as `LOC` (Location) or `O` (Outside), both `"New York"` and `"Los Angeles"` would become `LOC LOC LOC LOC`—making it impossible to tell whether that is one giant 4-word city or two separate 2-word cities! **BIO Tagging** prefixes the **first word** of every entity with `B-` (Beginning, e.g., `B-LOC`) and continuation words with `I-` (Inside, e.g., `I-LOC`), so `"New York Los Angeles"` becomes `B-LOC I-LOC B-LOC I-LOC`.
   - *Why Token-Level Accuracy is Useless for NER:* In normal English text, $> 92\%$ of words are not named entities (`O`). A lazy model that predicts `O` for every word in the test set gets 92% **token-level accuracy** while finding **zero** entities! That is why NER is always evaluated using **Exact Span-Level F1** (a predicted entity only counts as a True Positive if its start index, end index, *and* entity type match the ground truth 100%).
2. **Why Token-Level Softmax Fails on BIO & How Linear-Chain CRF Fixes It (`NLP15`):** If you put an independent Softmax classifier on top of each BiLSTM/BERT token state h_t, the decision at word t has **no idea what tag was actually chosen at word** $t-1$! If word 1 has 51% probability for `O` and 49% for `B-PER`, while word 2 has 51% probability for `I-PER`, independent greedy Softmax outputs **`O -> I-PER`**—which is **grammatically illegal** in BIO tagging (you cannot be *Inside* a Person entity without first *Beginning* one!).
   - A **Linear-Chain Conditional Random Field (CRF)** fixes this by learning a $K \times K$ **tag transition score matrix** A where entry $A_{i,j}$ rewards or penalizes transitioning from tag i at step $t-1$ to tag j at step t. The CRF very quickly learns $A_{\text{O,I-PER}} \approx -\infty$ and scores the **entire sequence of tags** $y_{1:T}$ **globally**. Because checking all K^T tag sequences by brute force is impossible (10^{50} paths for $K = 10, T = 50$), CRF uses the **Forward Algorithm** during training and **Viterbi Dynamic Programming** at inference to find the exact best path in $O(T \cdot K^2)$ time!
3. **Why CTC Loss Exists for Speech & OCR (`NLP20`):** In NER, 10 input words map cleanly to 10 output tags ($T = U$). But in speech recognition, you might have $T = 300$ audio frames (10 ms each) for the 3-letter word `"too"` ($U = 3$), and nobody labeled which exact millisecond corresponds to `'t'` vs. `'o'`! **Connectionist Temporal Classification (CTC)** lets the network output one character (or a special blank symbol `'-'`) at every single frame, and then collapses the T-frame output into a word using a simple 2-step rule: **(1) merge consecutive identical characters first, then (2) delete all blanks `'-'`**. Because consecutive `'o'`s get merged into a single `'o'`, to spell a word with a true double letter like `"too"`, the model **must** emit at least one blank `'-'` between the two `'o'`s (e.g., `'t-o-o'`)!



Linear-Chain CRF Viterbi Trellis vs. Illegal Greedy BIO Decoding

📐 The Math & Every Symbol Explained

🔬 Deep-Dive Math: Linear-Chain CRF Partition Function Z(X) & Viterbi DP Equations

1. **Linear-Chain CRF Global Sequence Score & Probability (`NLP15`):**

$$s(X, y_{1:T}) = \sum_{t=1}^{T} (E_{t,y_t} + A_{y_{t-1},y_t})$$

$$P(y_{1:T} \mid X) = \frac{\exp(s(X, y_{1:T}))}{Z(X)}, \qquad Z(X) = \sum_{y' \in \mathcal{Y}^T} \exp(s(X, y'))$$

- T: Sequence length (number of tokens).
- $K = |\mathcal{Y}|$: Number of distinct BIO tags in the tagset $\mathcal{Y}$.
- $E \in \mathbb{R}^{T \times K}$ (T **rows,** K **columns**): **Emission score matrix** output by the encoder (e.g., BiLSTM or BERT linear head), where $E_{t,k}$ is the local logit score for assigning tag k to token t.
- $A \in \mathbb{R}^{K \times K}$ (K **rows,** K **columns**): Learnable **Transition score matrix**, where $A_{i,j}$ is the score of transitioning from tag i at step $t-1$ to tag j at step t.
- $Z(X)$: **Partition function** summing $\exp(s(X, y'))$ over all K^T possible tag sequences $y' \in \mathcal{Y}^T$. During training, we minimize the negative log-likelihood $-\ln P(y^*_{1:T} \mid X) = \ln Z(X) - s(X, y^*_{1:T})$, computing $\ln Z(X)$ in $O(T \cdot K^2)$ time via the **Forward Algorithm** (which replaces max with logsumexp).

2. **Viterbi Dynamic Programming Recurrence (Exact Inference in $O(T \cdot K^2)$ Time):**

$$V_t(k) = E_{t,k} + \max_{j \in \{1,\dots,K\}} (V_{t-1}(j) + A_{j,k})$$

- $V_t(k)$: Score of the highest-scoring partial tag path ending in tag k at time step t (stored in a trellis matrix $V \in \mathbb{R}^{T \times K}$ of T **rows,** K **columns**).
- At each step $t \in \{1, \dots, T\}$ and for each target tag $k \in \{1, \dots, K\}$, we check all K previous tags $j \in \{1, \dots, K\}$ ($\implies K \times K = K^2$ operations per step, or $O(T \cdot K^2)$ total!).

- **CTC Alignment Collapse Function $\mathcal{B}(\pi)$ (`NLP20`):**

$$\mathcal{B} : (\Sigma \cup \{\text{'-'}\})^T \to \Sigma^{\leq T}$$

- **Step 1:** Collapse any contiguous run of identical symbols into a single symbol (e.g., `'tt--oo'` → `'t-o'`).
- **Step 2:** Remove all blank tokens `'-'` (e.g., `'t-o'` → `'to'`).

<details>
<summary>📌 Optional / Speech & OCR Bonus (Skip unless reviewing NLP20): Worked CTC Collapse Examples</summary>

Why is the order of operations in CTC (**Step 1: merge duplicates** → **Step 2: drop blanks**) so important? Let's trace three 6-frame alignments for target words `"to"`, `"too"`, and `"tto"`:

- **Alignment 1:** `'tt--oo'`
  - Step 1 (merge consecutive duplicates): `'tt'` → `'t'`, `'--'` → `'-'`, `'oo'` → `'o'` $\implies$ `'t-o'`
  - Step 2 (drop all blanks `'-'`): **`'to'`** *(Notice that `'oo'` without a blank between them collapsed into a single `'o'`!)*
- **Alignment 2:** `'tto-o-'`
  - Step 1 (merge consecutive duplicates): `'tt'` → `'t'` $\implies$ `'to-o-'`
  - Step 2 (drop all blanks `'-'`): **`'too'`** *(Because there was a blank `'-'` separating the two `'o'`s, Step 1 did NOT merge them, so Step 2 leaves `'oo'` intact!)*
- **Alignment 3:** `'t-t-oo'`
  - Step 1 (merge consecutive duplicates): `'oo'` → `'o'` $\implies$ `'t-t-o'`
  - Step 2 (drop all blanks `'-'`): **`'tto'`**

</details>

🔢 Step-by-Step Worked Toy Example

Viterbi vs. Greedy Decoding on a 2-Step Toy NER Trellis (`NLP15`) Suppose our input is a $T = 2$ word phrase: $x_1 =$ "New", $x_2 =$ "York", and we have $K = 3$ BIO tags: `{1: B-LOC, 2: I-LOC, 3: O}`. Suppose the encoder produces the following **Emission Matrix** $E \in \mathbb{R}^{2\times 3}$ **(2 rows, 3 columns)** (assuming uniform start transitions $A_{\text{START},k} = 0$):

Time Step / Token	$E_{t,\text{B-LOC}}$	$E_{t,\text{I-LOC}}$	$E_{t,\text{O}}$
$t = 1$ (`"New"`)	2.0	0.0	**2.2** *(slightly prefers `'O'` because `"New"` is a common adjective!)*
$t = 2$ (`"York"`)	1.0	**3.0**	0.5

And suppose the CRF has learned the **Transition Matrix** $A \in \mathbb{R}^{3\times 3}$ **(3 rows = from-tag i, 3 columns = to-tag j)**:

- $A_{\text{B-LOC}\to\text{I-LOC}} = +\mathbf{2.0}$ (strongly rewards valid multi-word location continuation!)
- $A_{\text{O}\to\text{I-LOC}} = -\mathbf{10.0}$ (heavily penalizes the illegal `O -> I-LOC` transition!)
- All other transitions ($A_{\text{B-LOC}\to\text{B-LOC}}, A_{\text{B-LOC}\to\text{O}}, A_{\text{O}\to\text{O}}, A_{\text{O}\to\text{B-LOC}}$) $= 0.0$.

1. **What does Independent Token-Level Greedy Decoding do?**
   - At $t = 1$ (`"New"`), it looks only at row 1 of E: $\arg\max(2.0, 0.0, 2.2) =$ O (score 2.2).
   - At $t = 2$ (`"York"`), it looks only at row 2 of E: $\arg\max(1.0, 3.0, 0.5) =$ I-LOC (score 3.0).
   - **Greedy Output:** **`O -> I-LOC`** (an **illegal** BIO sequence! Its true global CRF score is $2.2 - 10.0 + 3.0 = -4.8$).
2. **What does CRF Viterbi Dynamic Programming do?**

- **Step $t = 1$ (`"New"`):** Initialize $V_1(k) = E_{1,k}$:

  $$V_1(\text{B-LOC}) = 2.0, \qquad V_1(\text{I-LOC}) = 0.0, \qquad V_1(\text{O}) = 2.2$$

- **Step $t = 2$ (`"York"`):** Compute $V_2(\text{I-LOC}) = E_{2,\text{I-LOC}} + \max_j[V_1(j) + A_{j \to \text{I-LOC}}]$:
  - From `j = B-LOC`: $V_1(\text{B-LOC}) + A_{\text{B-LOC} \to \text{I-LOC}} = 2.0 + 2.0 = \mathbf{4.0}$ *(Best predecessor!)*
  - From `j = I-LOC`: $V_1(\text{I-LOC}) + A_{\text{I-LOC} \to \text{I-LOC}} = 0.0 + 0.0 = 0.0$
  - From `j = O`: $V_1(\text{O}) + A_{\text{O} \to \text{I-LOC}} = 2.2 + (-10.0) = -7.8$
  - Add emission $E_{2,\text{I-LOC}} = 3.0$:

    $$V_2(\text{I-LOC}) = 3.0 + 4.0 = \mathbf{7.0}$$

- Compare with $V_2(\text{B-LOC}) = 1.0 + \max(2.0 + 0, 0, 2.2 + 0) = 3.2$ and $V_2(\text{O}) = 0.5 + 2.2 = 2.7$.
- **Viterbi Backtrace:** The highest score at $t = 2$ is $V_2(\text{I-LOC}) = 7.0$, and its backpointer points to `B-LOC` at $t = 1$. Thus Viterbi outputs the globally optimal, grammatically valid span **`B-LOC -> I-LOC`**!

🎯 Exam Takeaway & Common Traps

- **CRF Complexity Trap (`NLP15`):** With T tokens and K tags, brute-force search is $O(K^T)$, whereas both **Forward training** and **Viterbi inference** take $O(T \cdot K^2)$ time (quadratic in the number of tags K, linear in sequence length T!).
- **CTC Double-Letter Trap (`NLP20`):** Because CTC merges consecutive identical tokens *before* deleting blanks `'-'`, emitting a true repeated character (like `'ll'` in `"hello"` or `'oo'` in `"too"`) **strictly requires at least one blank `'-'` between the two identical letters**.

2.3 🌟 [Core — Bread & Butter] Seq2Seq, Attention Variants, Decoding & NLP Evaluation Metrics

💡 Intuition (Plain English First)

How do Encoder-Decoder (Seq2Seq) models translate a French sentence into English or summarize an article, how do we search for the best output sentence, and how do we grade the result?

1. **The Seq2Seq Bottleneck & How Bahdanau/Luong Attention Fixed It (`NLP13`):** In early RNN Seq2Seq models (Sutskever et al., 2014), the Encoder read a 50-word source sentence $x_1, \ldots, x_{T_x}$ and squashed its *entire meaning* into the **single final hidden state vector** h_{T_x}—like trying to write the summary of a whole novel on the back of a postage stamp! By the time the Decoder was generating word 30, details from word 1 had vanished.
   - **The Attention Fix (Bahdanau 2014, Luong 2015):** Instead of throwing away intermediate encoder states, keep **all** encoder hidden states $h_1, h_2, \ldots, h_{T_x}$ on a shelf! At every decoder step t, the decoder uses its current state s_t as a "search query" to compare against every encoder state h_i, turns those compatibility scores $e_{t,i}$ into a probability distribution $\alpha_{t,i}$ via Softmax, and computes a fresh **context vector** $c_t = \sum_i \alpha_{t,i} h_i$ focused on whichever source words are being translated *right now*.
   - **Bahdanau (Additive) vs. Luong (Multiplicative):** Bahdanau passes s_t and h_i through a small 1-layer neural network ($v_a^T \tanh(W_1 s_t + W_2 h_i)$), whereas Luong computes a fast matrix/dot product ($s_t^T W_a h_i$ or

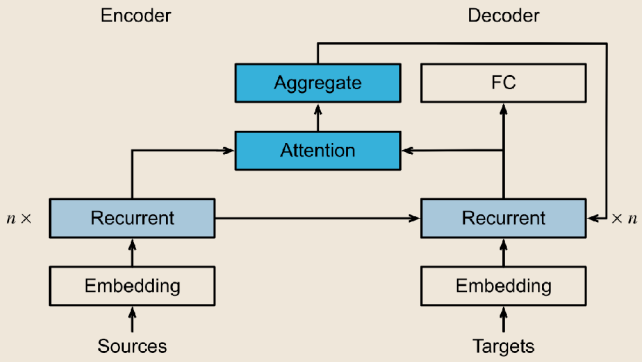
$s_t^T h_i$). Later, **Vaswani et al. (2017)** divided the dot product by $\sqrt{d_k}$ so large dot products wouldn't push Softmax into flat, zero-gradient saturation!

2. **Why Teacher Forcing Causes Exposure Bias (`NLP12`):** When training an autoregressive decoder, what token should we feed as input at step t? If we feed the model's own (initially terrible) predictions, a mistake at word 1 derails the rest of the sentence, making training slow and unstable. **Teacher Forcing** fixes this during training by always feeding the **ground-truth human word** y_{t-1}^* as the input to step t.
   - **The Catch (Exposure Bias):** At test time, there is no human teacher! The model is forced to feed its **own predicted words** $\hat{y}_{t-1}$ back into itself. Because the model was *only ever exposed* to perfect human prefixes during training, a single slightly odd word choice at test time pushes the model into an unfamiliar state, causing errors to snowball rapidly over long generations.

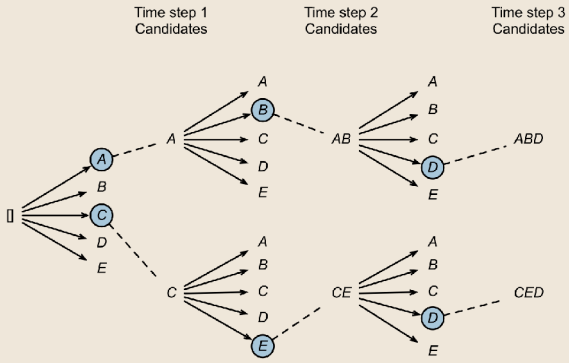
3. **Why Beam Search Needs a Length Penalty (`NLP14`):** At each decoding step, **Greedy Decoding ($B = 1$)** picks the single highest-probability word right now, which can easily walk into a grammatical trap two steps later. **Beam Search** keeps the top B (say $B = 4$) most promising partial sentences alive at every step.
   - **Why Unnormalized Beam Search Loves Short Sentences:** Every word probability $P(y_t \mid y_{<t}, X)$ is a number ≤ 1, so its log-probability $\ln P(y_t \mid \dots)$ is **strictly negative** (e.g., -1.5). Therefore, every time a candidate sentence adds one more word, its total score $\sum_{t=1}^{L} \ln P$ **gets more negative (decreases)**! Without correction, a short 3-word fragment beats a complete 15-word translation every time. Dividing the cumulative log-probability by L^α ($\alpha \in [0.6, 1.0]$) computes the *average* log-probability per token, leveling the playing field between short and long sentences.

4. **Why BLEU Needs Clipped Precision & a Brevity Penalty (`NLP16`):** How do we automatically score a machine translation C against a human reference R?
   - **Why Clipped Precision?** Suppose the reference is `"the cat is on the mat"` (6 words) and a broken model outputs `"the the the the"` (4 words). Naive unigram precision says: "*Every `'the'` in the candidate appears in the reference, so precision* $= 4/4 = 100\%$*!*" **Clipped Precision** stops this exploit by capping each word's numerator credit at the maximum number of times it appears in the reference ($\text{count}_R(\text{"the"}) = 2 \implies 2/4 = 50\%$).
   - **Why the Brevity Penalty (BP)?** Even with clipping, what if the model outputs a 1-word translation: just `"the"`? Now clipped count $= 1$ and candidate length $c = 1$, giving $p_1 = 1/1 = 100\%$ precision! Why doesn't BLEU just use Recall like F1? Because in translation, you often have *multiple valid human references* (`"the cat is on the mat"` vs. `"there is a feline atop the rug"`), and a good translation shouldn't be forced to recall words from *all* references at once! Instead, BLEU replaces recall with a **Brevity Penalty** $\text{BP} = \exp(1 - r/c)$ whenever the candidate is shorter than the reference ($c \leq r$). If $c > r$, $\text{BP} = 1$ (since clipped precision already penalizes overly long outputs).

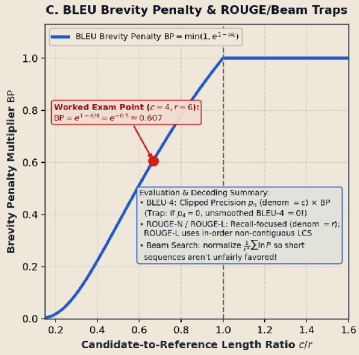
5. **Why Summarization Uses ROUGE-L & Semantic Eval Uses BERTScore (`NLP17`):**
   - In summarization, we care most about **Recall**: *Did the summary capture the key facts from the reference?* **ROUGE-N** divides matched n-grams by the **Reference length** (not Candidate length!), and **ROUGE-L** measures the **Longest Common Subsequence (LCS)**—words appearing in the same relative order, even if other words are inserted in between!
   - Both BLEU and ROUGE are purely lexical (exact string match): if the reference says `"automobile"` and the model writes `"car"`, BLEU and ROUGE give 0 **credit**! **BERTScore** fixes this by matching token embeddings via cosine similarity in a pretrained Transformer space.



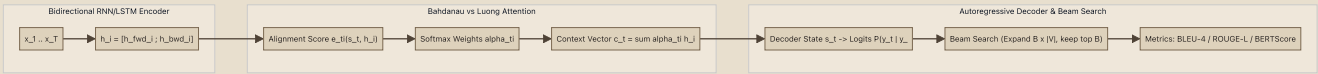
Seq2Seq Encoder-Decoder with Bahdanau Additive Attention



Beam Search Candidate Expansion Across Decoding Time Steps



BLEU Clipped Precision, Brevity Penalty & ROUGE-L Recall



📐 The Math & Every Symbol Explained

1. **Seq2Seq Attention Equations & Alignment Variants (`NLP13`):** Let encoder hidden states be $h_1, \dots, h_{T_x} \in \mathbb{R}^{d_h}$ and the decoder state at step t be $s_t \in \mathbb{R}^{d_s}$:

$$e_{t,i} = \text{score}(s_t, h_i), \qquad \alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^{T_x} \exp(e_{t,j})}, \qquad c_t = \sum_{i=1}^{T_x} \alpha_{t,i} h_i$$

   - $e_{t,i} \in \mathbb{R}$: Raw unnormalized alignment score between decoder step t and encoder position i.

- $\alpha_{t,i} \in (0, 1)$: Normalized attention weight ($\sum_{i=1}^{T_x} \alpha_{t,i} = 1$).
- $c_t \in \mathbb{R}^{d_h}$: Dynamic context vector at decoder step t.

Attention Mechanism	Alignment Score Formula $\text{score}(s_t, h_i)$	Learnable Parameter Matrix Shapes ($n \times m$ = n rows, m cols) & Properties
Bahdanau (Additive, 2014)	$v_a^T \tanh(W_1 s_t + W_2 h_i)$	$W_1 \in \mathbb{R}^{d_a \times d_s}$ (d_a rows, d_s cols), $W_2 \in \mathbb{R}^{d_a \times d_h}$ (d_a rows, d_h cols), $v_a \in \mathbb{R}^{d_a \times 1}$. Works even when $d_s \neq d_h$!
Luong General (Multiplicative, 2015)	$s_t^T W_a h_i$	$W_a \in \mathbb{R}^{d_s \times d_h}$ (d_s rows, d_h cols). Fast bilinear matrix product; handles $d_s \neq d_h$.
Luong Dot-Product (2015)	$s_t^T h_i$	**Zero** learnable attention parameters! Strictly requires $d_s = d_h$.
Scaled Dot-Product (Vaswani 2017)	$\frac{s_t^T h_i}{\sqrt{d_k}}$	Divides by $\sqrt{d_k}$ so that if s_t, h_i have unit variance, $\text{Var}(s_t^T h_i) = d_k$ is rescaled back to 1, preventing Softmax saturation.

2. **Beam Search Length-Normalized Scoring Objective (`NLP14`):**

$$\text{Score}_{\text{norm}}(y_{1:L}) = \frac{1}{L^\alpha} \sum_{t=1}^{L} \ln P(y_t \mid y_{<t}, X)$$

- B: Beam width (number of active partial hypotheses kept at each step; $B = 1$ reduces to **Greedy Decoding**).
- $|V|$: Vocabulary size; at each step $t \in \{1, \dots, T\}$, Beam Search expands $B \times |V|$ candidates and keeps the top B, taking $O(T \cdot B \cdot |V|)$ time.
- L: Length (number of tokens) of candidate hypothesis $y_{1:L}$.
- $\alpha \in [0, 1]$: Length normalization exponent ($\alpha = 0$ is unnormalized; $\alpha = 1.0$ is exact per-token average log-probability; Google NMT uses $\frac{(5+L)^\alpha}{(5+1)^\alpha}$ with $\alpha \in [0.6, 0.7]$).

3. **BLEU-N (Bilingual Evaluation Understudy) Exact Formula (`NLP16`):**

$$\text{BLEU-}N = \text{BP} \cdot \exp\left(\sum_{n=1}^{N} w_n \ln p_n\right) = \text{BP} \cdot \prod_{n=1}^{N} p_n^{w_n}, \qquad \text{BP} = \begin{cases} 1 & \text{if } c > r \\ \exp\left(1 - \frac{r}{c}\right) & \text{if } c \leq r \end{cases}$$

$$p_n = \frac{\sum_{g \in \text{Unique } n\text{-grams in } C} \min\left(\text{count}_C(g),\ \max_{R \in \text{Refs}} \text{count}_R(g)\right)}{\text{Total number of } n\text{-grams in Candidate } C}$$

- $c = |C|$: Token length of the Candidate translation.
- $r = |R|$: Effective Reference translation length.
- p_n: Clipped n-gram **Precision** (denominator is the **Candidate** n-gram count $c - n + 1$).
- w_n: Positive weights summing to 1 (for standard **BLEU-4**, $N = 4$ and $w_1 = w_2 = w_3 = w_4 = \frac{1}{4} = 0.25$).

4. **ROUGE-N & ROUGE-L (Recall-Oriented Understudy for Gisting Evaluation, `NLP17`):**

$$\text{ROUGE-}N = \frac{\sum_{g \in R} \min(\text{count}_C(g), \text{count}_R(g))}{\text{Total number of } n\text{-grams in Reference } R}$$

$$R_{\text{LCS}} = \frac{\text{LCS}(R, C)}{|R|}, \qquad P_{\text{LCS}} = \frac{\text{LCS}(R, C)}{|C|}, \qquad F_{\text{LCS}} = \frac{(1+\beta^2) R_{\text{LCS}} P_{\text{LCS}}}{R_{\text{LCS}} + \beta^2 P_{\text{LCS}}}$$

- $\text{LCS}(R, C)$: Length of the **Longest Common Subsequence** between Reference R and Candidate C (tokens must appear in the same relative order, **not necessarily contiguous**!).
- For balanced F1 score ($\beta = 1$), $F_{\text{LCS}} = \frac{2 R_{\text{LCS}} P_{\text{LCS}}}{R_{\text{LCS}} + P_{\text{LCS}}}$.

▼ 📌 **Optional Deep-Dive (Skip for Timed MCQ): ChrF++ & Self-BLEU Diversity Metrics**

- **ChrF / ChrF++ (Popović, 2015/2017):** Computes character n-gram (typically $n \in [1, 6]$) Precision and Recall F-score (with word n-grams added in ChrF++). Because it operates at the character level, ChrF++ rewards morphologically related words (e.g., `"translating"` vs. `"translated"`) and is completely immune to differences in subword tokenization.
- **Self-BLEU (Zhu et al., 2018):** How do you measure whether a generative LLM is suffering from **mode collapse** (repeating the same generic phrases over and over)? Generate M sample outputs, treat each output s_i as a candidate, and compute its BLEU score against the remaining $M - 1$ generated outputs as references! **Lower Self-BLEU = higher generation diversity**; high Self-BLEU indicates repetitive mode collapse.

🔢 Step-by-Step Worked Toy Example

Example 1: Attention Step Walkthrough (Scores → Softmax → Context Vector) Suppose our Encoder has processed a 3-word input ($T_x = 3$) with 2D hidden states:

$$h_1 = \begin{bmatrix} 1.0 \\ 0.0 \end{bmatrix}, \quad h_2 = \begin{bmatrix} 0.0 \\ 2.0 \end{bmatrix}, \quad h_3 = \begin{bmatrix} 3.0 \\ 1.0 \end{bmatrix}$$

At decoder step $t = 1$, suppose the alignment scores between s_1 and h_1, h_2, h_3 are $e_1 = [\ln 2,\, \ln 1,\, \ln 1] = [0.693,\, 0.0,\, 0.0]$.

1. **Compute Softmax Attention Weights $\alpha_{1,i}$:**

$$\exp(e_1) = [e^{\ln 2},\, e^0,\, e^0] = [2,\, 1,\, 1], \qquad \sum_{j=1}^{3} \exp(e_{1,j}) = 2 + 1 + 1 = 4$$

$$\alpha_1 = \left[\frac{2}{4}, \frac{1}{4}, \frac{1}{4}\right] = [\mathbf{0.50}, \mathbf{0.25}, \mathbf{0.25}]$$

2. **Compute Context Vector $c_1 = \sum_{i=1}^{3} \alpha_{1,i} h_i$:**

$$c_1 = 0.50 \begin{bmatrix} 1.0 \\ 0.0 \end{bmatrix} + 0.25 \begin{bmatrix} 0.0 \\ 2.0 \end{bmatrix} + 0.25 \begin{bmatrix} 3.0 \\ 1.0 \end{bmatrix} = \begin{bmatrix} 0.50 + 0 + 0.75 \\ 0 + 0.50 + 0.25 \end{bmatrix} = \begin{bmatrix} 1.25 \\ 0.75 \end{bmatrix}$$

Example 2: Beam Search Length Normalization Flipping the Winner (`NLP14`) Suppose Beam Search ($B = 2$) is comparing two completed candidate sentences:

- **Candidate A (short, $L_A = 2$ tokens):** $\ln P(y_1) = -1.2$, $\ln P(y_2 \mid y_1) = -1.2 \implies$ Unnormalized sum $= \mathbf{-2.4}$.
- **Candidate B (complete, $L_B = 6$ tokens):** Each of the 6 tokens has $\ln P(y_t \mid y_{<t}) = -0.8 \implies$ Unnormalized sum $= 6 \times (-0.8) = \mathbf{-4.8}$.

- **Without Length Normalization ($\alpha = 0$):** Candidate A wins because $-2.4 > -4.8$, even though Candidate B had much higher confidence per word (-0.8 vs. -1.2)!
- **With Length Normalization ($\alpha = 1.0$):**

$$\text{Score}_{\text{norm}}(A) = \frac{-2.4}{2^1} = -1.20 < \text{Score}_{\text{norm}}(B) = \frac{-4.8}{6^1} = \mathbf{-0.80} \quad \text{(Candidate B rightly wins!)}$$

Example 3: BLEU Clipped Precision & Brevity Penalty Calculations (`NLP16`)

- **Case A (Clipped Unigram Precision & BP):**
  - Candidate C: `"the cat the cat"` (length $c = 4$)
  - Reference R: `"the cat is on the mat"` (length $r = 6$)
  - **Step 1 (Clipped Counts):**
    - Word `"the"`: appears $\text{count}_C = 2$ times in Candidate, $\text{count}_R = 2$ times in Reference $\implies \min(2, 2) = \mathbf{2}$.
    - Word `"cat"`: appears $\text{count}_C = 2$ times in Candidate, $\text{count}_R = 1$ time in Reference $\implies \min(2, 1) = \mathbf{1}$.
  - **Step 2 (Clipped Unigram Precision p_1):** Divide by **Candidate length** $c = 4$:

$$p_1 = \frac{2 + 1}{4} = \frac{3}{4} = \mathbf{0.75}$$

  - **Step 3 (Brevity Penalty BP):** Since $c = 4 \leq r = 6$:

$$\text{BP} = \exp\left(1 - \frac{r}{c}\right) = \exp\left(1 - \frac{6}{4}\right) = \exp(1 - 1.5) = \mathbf{e^{-0.5}} \approx \mathbf{0.6065}$$

- **Case B (Half-Length Candidate BP):** What if a candidate has length $c = 6$ and the reference has length $r = 12$?

$$\text{BP} = \exp\left(1 - \frac{12}{6}\right) = \exp(1 - 2) = \mathbf{e^{-1}} \approx \mathbf{0.3679}$$

What if the candidate has length $c = 15$ and the reference has length $r = 12$ ($c > r$)? Then BP $= 1.0$ (never > 1!).

Example 4: ROUGE-L Longest Common Subsequence Calculation (`NLP17`)

- Reference R ($|R| = 4$): `"police killed the gunman"`
- Candidate C ($|C| = 6$): `"police ended up killing the gunman"`
- **Step 1 (Find LCS):** The words `"police"`, `"the"`, `"gunman"` appear in both R and C in that exact left-to-right order (even though `"ended up killing"` sits between `"police"` and `"the"`!). Thus $\text{LCS}(R, C) = \mathbf{3}$.
- **Step 2 (Compute Recall, Precision, and F1):**

$$R_{\text{LCS}} = \frac{3}{|R|} = \frac{3}{4} = \mathbf{0.75}, \qquad P_{\text{LCS}} = \frac{3}{|C|} = \frac{3}{6} = \mathbf{0.50}, \qquad F_1 = \frac{2 \times 0.75 \times 0.50}{0.75 + 0.50} = \frac{0.75}{1.25} = \mathbf{0.60}$$

🎯 Exam Takeaway & Common Traps

- **BLEU vs. ROUGE Denominator Trap (`NLP16`, `NLP17`): BLEU is Precision-based** (divides matched n-grams by **Candidate length** c, plus a Brevity Penalty for recall), whereas **ROUGE-N is Recall-based** (divides matched n-grams by **Reference length** r).
- **BLEU-4 Zero-Count Geometric Mean Trap (`NLP16`):** Because standard BLEU-4 takes the geometric mean $\text{BP} \cdot (p_1 p_2 p_3 p_4)^{1/4}$, if **any single n-gram order has 0 matches** (for instance, $p_4 = 0$ on a 3-word translation or when no 4-gram matches exactly), the entire unsmoothed BLEU-4 **score equals** 0! (In practice, sentence-level BLEU uses **Chen & Cherry smoothing** to avoid zeroing out).
- **ROUGE-L Contiguity Trap (`NLP17`):** The Longest Common Subsequence (LCS) in ROUGE-L only requires matched words to appear in the **same relative order**, **NOT** adjacently/contiguously!
- **Teacher Forcing vs. Exposure Bias (`NLP12`):** Teacher Forcing is the *training technique* (feeding ground-truth y^*_{t-1}); **Exposure Bias** is the resulting *train-test discrepancy* at inference time when the model must condition on its own imperfect predictions $\hat{y}_{<t}$.

2.4 Rapid-Fire 60-Second Exam Checklist (All 20 `NLP01` – `NLP20` Targets)

Target ID	Concept Tested	10-Second Formula / Rule to Remember
`NLP01`	BPE vs. WordPiece Merge & Inference	BPE merges max raw $\text{count}(ab)$ & applies rules in **training order**; WordPiece merges max PMI $\frac{\text{count}(ab)}{\text{count}(a)\text{count}(b)}$ & runs greedy `MaxMatch`.
`NLP02`	Exact BPE Vocabulary Size	$\lvert V\rvert_{\text{BPE}} = \lvert\text{Base Alphabet}\rvert + N_{\text{merges}} + \lvert\text{Special Tokens}\rvert$ (256 base bytes in Byte-Level BPE $\implies$ 0 `<UNK>`).
`NLP03`	Add-k / Laplace Smoothing	$P_{\text{Add-}k}(w_t \mid w_{t-1}) = \frac{\text{count}(w_{t-1}, w_t) + k}{\text{count}(w_{t-1}) + k\lvert\mathbf{V}\rvert}$ (denominator adds $k\lvert V\rvert$, never k).
`NLP04`	Perplexity (`PPL`) & Log Bases	$\text{PPL} = e^{\mathcal{L}_{\text{CE, nats}}} = 2^{\mathcal{L}_{\text{CE, bits}}}$; uniform random model gives $\text{PPL} = \lvert V\rvert$; not comparable across different $\lvert V\rvert$.
`NLP05`	TF-IDF & BM25 Hyperparameters	Stopword in all N docs $\implies$ $\text{IDF} = \log(1) = 0$. In BM25, k_1 caps TF saturation at $k_1 + 1$; $b \in [0, 1]$ controls length normalization ($b = 0$ disables it).
`NLP06`	Word2Vec Parameter Count	Maintains both $W_{\text{in}} \in \mathbb{R}^{\lvert V\rvert \times d}$ and $W_{\text{out}} \in \mathbb{R}^{\lvert V\rvert \times d}$ $\implies$ $\mathbf{2\lvert V\rvert d}$ total parameters; Skip-Gram excels on **rare words**.
`NLP07`	Softmax Approximations	Full Softmax $= O(\lvert V\rvert)$; Hierarchical Softmax (Huffman tree) $= O(\log_2 \lvert V\rvert)$; Negative Sampling (SGNS) $= O(K)$ binary sigmoids.
`NLP08`	Unit-Vector Cosine vs. L_2 Distance	When $\lVert u\rVert_2 = \lVert v\rVert_2 = 1$, $\lVert u - v\rVert_2 = \sqrt{2(1 - \cos\theta)}$ (min L_2 ranking $\equiv$ max Cosine ranking).
`NLP09`	RNN / GRU / LSTM / BiLSTM Params	Let $U = h(d + h + 1)$: **RNN** $= 1U$, **GRU** $= 3U$, **LSTM** $= 4U$, **1-Layer BiLSTM** $= 8U$ (Layer 2 of BiLSTM takes $d_2 = 2h$).
`NLP10`	LSTM Gate Equations & Highway	$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ (additive cell state highway prevents vanishing gradients; initialize $b_f = +1.0$).
`NLP11`	Vanishing vs. Exploding Gradients	$\sigma_{\max}(W_{hh}) > 1 \implies$ Exploding (fix: **Gradient Norm Clipping**); $\sigma_{\max} < 1 \implies$ Vanishing (fix: **LSTM/GRU highway** or **Transformer**).

Target ID	Concept Tested	10-Second Formula / Rule to Remember
`NLP12`	Teacher Forcing & Exposure Bias	Teacher Forcing feeds ground-truth y^*_{t-1} at training; causes **Exposure Bias** at test time when conditioning on self-generated $\hat{y}_{<t}$.
`NLP13`	Bahdanau vs. Luong Attention	Bahdanau (Additive): $v_a^T \tanh(W_1 s_t + W_2 h_i)$; Luong (Multiplicative): $s_t^T W_a h_i$ or $s_t^T h_i$; Vaswani scales by $1/\sqrt{d_k}$.
`NLP14`	Beam Search Length Penalty	Complexity $O(T \cdot B \cdot \lvert V \rvert)$; unnormalized sum $\sum \ln P < 0$ penalizes long sequences $\implies$ divide by L^α.
`NLP15`	Linear-Chain CRF & Viterbi	Adds $K \times K$ tag transition matrix A to prevent illegal BIO sequences (`O -> I-PER`); Forward & Viterbi run in $O(T \cdot K^2)$ time.
`NLP16`	BLEU-4 & Brevity Penalty (`BP`)	Precision-focused (divides by Candidate length c); BP $= \exp(1 - r/c)$ if $c \leq r$ else 1; if any $p_n = 0$, unsmoothed BLEU-4 $= 0$.
`NLP17`	ROUGE-N & ROUGE-L	Recall-focused (divides by Reference length r); ROUGE-L uses in-order (non-contiguous) Longest Common Subsequence $\text{LCS}(R, C)$.
`NLP18`	Levenshtein Edit Distance	$O(M \cdot N)$ Wagner-Fischer DP over Insertions, Deletions, Substitutions (`'INTENTION'` $\rightarrow$ `'EXECUTION'` $= 5$ edits for unit cost, 8 if sub $= 2$).
`NLP19`	BERT MLM 80-10-10 Rule	15% of tokens selected for loss: 80% `[MASK]`, 10% **random token**, 10% **unchanged** (mitigates pretrain-finetune discrepancy).
`NLP20`	CTC Loss Collapse Rules	**(1) Merge consecutive duplicates first, (2) drop all blanks** `'-'` (a true double letter like `'oo'` in `'too'` requires `'o-o'`).